

Construire la partie LabVIEW pas à pas

Recette complète de construction et de vérification

LabVIEW 2025 Q3 · menus anglais · Windows / VM EPFL

Louis Pédelaborde, Romeo Mugnier de Almeida et Raphaël Pical

Version du 24 septembre 2026

Suivez les chapitres 1 à 7, puis l'annexe G au chapitre 8. Reprenez ensuite au chapitre 9 et poursuivez jusqu'aux contrôles finaux.

Sommaire

- Construire la partie LabVIEW du projet Binairo 4
- Mode d’emploi de cette recette 5
- 1 Préparer le dossier une seule fois 6
- 2 Les gestes LabVIEW à utiliser partout 8
- 3 Créer les types communs 11
- 4 ErreurSi.vi 12
- 5 LireTexte.vi et EcrireTexte.vi 13
- 6 SupprimerResultat.vi 14
- 7 LireImage.vi 15
- 8 Géométrie de la grille 18
- 9 EcrireCellule.vi 19
- 10 CommandeOCR.vi 21
- 11 AnalyserResultatOCR.vi 22
- 12 LireCase.vi 24
- 13 GenererScript.vi 26
- 14 Option facultative : inspecter le lanceur fourni 28
- 15 LancerMatlab.vi 29
- 16 VerifierPDF.vi 31
- 17 BinairoSolver.vi le programme principal 32
- 18 Lire les résultats attendus 35
- 19 Batterie finale de validation native 37
- 20 Si quelque chose ne marche pas 39
- 21 Terminer et préparer la remise ultérieure 40

22 Références à consulter en cas de différence	41
23 Contrôle de couverture avant de déclarer la partie terminée	42
Annexe G — Détecter la grille et découper ses cases	44
G0. Conventions à appliquer à chaque fil	45
G1. Vérifier les deux types et attribuer les connecteurs	46
G2. EtendueNoire.vi — trouver le premier et le dernier pixel occupé	47
G3. IntervallesNoirs.vi — relever tous les traits, même aux bords	49
G4. ComputeCellRect.vi — rectangle intérieur d'une case	51
G5. ComputeRowsCols.vi — nombre de cases et traits mesurés	53
G6. Essais géométriques sans image et extraction effective	56

Construire la partie LabVIEW du projet Binairo

Recette de construction pour Louis Pédelaborde, Romeo Mugnier de Almeida et Raphaël Pical. Version du 24 septembre 2026.

Ce document explique comment construire les VIs, les raccorder au C et à MATLAB, puis vérifier le résultat. Il s'adresse à une personne qui débute dans LabVIEW. Les consignes sont prévues pour **LabVIEW 2025 Q3, menus anglais, sous Windows**, notamment dans la VM EPFL. Les noms anglais ci-dessous sont ceux à rechercher dans LabVIEW ; les noms de vos propres objets sont en français. Ne construisez pas un projet Real-Time, FPGA ou NXG.

Référence prioritaire : Donnée projet Binairo 2026.2, fichier P00.PPI_Projet.2026.2.pdf, identifié sous ce titre dans l'export Moodle. Les exercices servent seulement d'aide compatible ; ils ne remplacent aucune consigne de la donnée. La recette conserve la contrainte « Code Matlab : Pas de boucles » des critères généraux de notation 2025.

Ce qui est établi : exigences de cette donnée, contrats des sources actuelles, principes et terminaux des fonctions NI référencées. **Ce qui reste à mesurer** : exécution des VIs que vous allez construire, interface du lanceur fourni seulement si vous choisissez cette option, et rendu MATLAB dans votre installation. Une recette ne remplace pas ces essais et ne garantit pas une note. Les dernières consignes Moodle restent à vérifier avant remise : le document 2026 annonce encore une liste d'erreurs à venir.

Règle OCR appliquée : la procédure détaillée de la donnée, page physique 18, impose $\text{score} \geq \text{seuil}$ pour les chiffres, puis le maximum de $\text{score} - \text{seuil}$. Elle prime ici sur le résumé « score brut » de sa page 10. Il existe donc une contradiction interne à la donnée, explicitement signalée ; le code suit désormais sa procédure détaillée. La case vide reste testée en premier avec $\text{ratio de blancs} > \text{seuil vide}$ (page 10). Si les marges sont exactement égales, conserver 0 est notre convention déterministe, faute de départage prescrit. Aucun connecteur principal définitif n'apparaît dans cette donnée : les connecteurs ci-dessous sont nos choix de construction, pas celui du Billard 2025.

Mise à jour de conformité : voir le contrôle contre la donnée, qui indique les corrections et les points de la donnée restant ambigus. Si vous avez déjà commencé, suivez sa dernière section pour adapter seulement les étapes concernées.

Mode d'emploi de cette recette

1. Ouvrez ce document à côté de LabVIEW. La recette est en français ; les noms des menus et des primitives restent en anglais.
2. Suivez les chapitres 1 à 7. Au chapitre 8, suivez toute l'annexe G, puis revenez au chapitre 9. Continuez ensuite jusqu'au chapitre 23.
3. Pour chaque sous-VI : créez d'abord la face avant, attribuez le connecteur, construisez le diagramme, puis exécutez le checkpoint. Ne raccordez pas un sous-VI dont le checkpoint échoue.
4. Une flèche cassée signifie que le câblage n'est pas terminé. Une flèche entière ne prouve pas que le résultat est correct : comparez toujours aux résultats indiqués.
5. À la fin d'une séance, sauvegardez tous les fichiers. Copiez le dossier Execution dans votre sauvegarde privée persistante. Les modifications faites seulement dans la VM ne sont pas automatiquement présentes dans GitHub.
6. Dans la version HTML, utilisez le sommaire pour vous déplacer et les boutons Copier pour les chaînes exactes. Les cases « Étape terminée et contrôlée » sont une aide personnelle : ne les cochez qu'après un essai réussi. Elles ne constituent pas une validation automatique.

La chaîne construite sera :

```
PNG → LireImage → ComputeRowsCols
      ↓
      pour chaque ligne, pour chaque colonne
      ComputeCellRect → Cell.bin → OCR.exe → CellValue.txt
      ↓
      matrice de 0 / 1 / NaN → solve.m
      ↓
      lanceur MATLAB → fonctions .m → PDF
```

Un seul dossier d'exécution, un seul nom de fichier par résultat, un seul fil d'erreur d'une étape à la suivante. Les VIs auxiliaires sont construits avant le programme principal ; vous n'avez pas d'architecture à choisir.

1 Préparer le dossier une seule fois

1. Démarrez la VM Windows. Ouvrez une session. Repérez un dossier personnel **persistant et inscriptible** : un fichier posé sur le bureau d'une VM peut être perdu lors d'une réinitialisation. Demandez au support EPFL si cette persistance n'est pas documentée. Ne changez pas l'installation logicielle du cours.
2. Dans l'Explorateur Windows, activez l'affichage des extensions de fichiers. Créez un dossier Binaire, puis un sous-dossier Execution. Les chemins donnés dans les exemples de commandes ne sont pas à graver dans les VIs.
3. Copiez dans Execution **les cinq fichiers** du dossier c de la livraison : `main.c`, `reconnaissance.c`, `reconnaissance.h`, `parametres_ocr.h`, `FontRasterized_0_1.h`. Le dernier est la police originale du professeur, incluse dans le ZIP de sources. Elle reste exclue du dépôt GitHub ; si vous partez d'un clone, reprenez-la dans le ZIP ou dans « fichiers utiles » sur Moodle. Ne créez pas une police de remplacement.
4. Copiez aussi les **treize fichiers** .m de la livraison dans Execution : `CheckValidMove`, `CheckVectorOk`, `CheckVectorUniqueOk`, `CheminPdfBinaire`, `CreerFigureBinaire`, `DirectValues`, `DispBinaire`, `DisplayBinaire`, `ExecuterBinaire`, `FormatGrilleValide`, `GrilleBinaireValide`, `SolveBinaire`, `printGrid`. Gardez l'extension .m et la casse exacte. `ExecuterBinaire` reste disponible pour les anciens appels, mais le script généré ci-dessous appelle directement `SolveBinaire` et `DisplayBinaire`.
5. Le mode normal de cette recette lance MATLAB directement par System Exec. **Facultatif** : copiez `MP_LaunchMatlabScript4.vi` depuis Moodle uniquement si vous souhaitez utiliser ce lanceur alternatif ; conservez alors ailleurs son original.
6. Dans Binaire, créez Images et Preuves. Copiez les PNG du cours dans Images, **jamais à la place d'un résultat**. En particulier : `Binaire_6x6.png`, `Binaire_8x8.png`, `Binaire_4x4_Bad.png` et `Binaro_5x6_Bad.png` si ce dernier est fourni. Le nom Binaro de cette dernière image n'est pas à corriger.
7. Ouvrez LabVIEW. Help > About LabVIEW : notez version et architecture dans Preuves/version.txt. Faites de même avec MATLAB (version dans sa fenêtre de commande). Ces notes décrivent votre installation, pas le programme à remettre.
8. Dans LabVIEW : File > Create Project > Blank Project, puis enregistrez Binaire.lvproj **dans** Execution. Tous les VIs et .ctl de cette recette sont enregistrés à cet endroit, sous My Computer. Ajoutez les fichiers avec clic droit My Computer > Add > File si nécessaire.
9. Une seule personne modifie les VIs à la fois. Ne faites pas tourner deux exemplaires du programme dans le même dossier : ils partageraient Cell.bin, CellValue.txt et solve.m.

Après préparation : tous les .vi, .ctl, .m, les sources C et l'exécutable seront côte à côte dans Execution. Les images peuvent rester dans Images. Ne créez pas de copie de travail supplémentaire à synchroniser à chaque changement.

1.1 Compiler le C dans Windows

Ouvrez le terminal du compilateur **fourni dans votre environnement de cours**. Placez-vous dans Execution avec `cd /d "chemin réel vers Execution"` dans l'invite Windows. Cette saisie manuelle n'est pas un chemin absolu intégré au code.

- Si `gcc --version` fonctionne, utilisez :

```
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror -Wvla main.c reconnaissance.c -o OCR.exe
```
- Si le cours utilise Visual Studio, ouvrez son **Developer Command Prompt**, puis utilisez :

```
cl /nologo /std:c11 /W4 /TC main.c reconnaissance.c /Fe:OCR.exe
```

N'exécutez qu'une des deux commandes, selon le compilateur réellement installé. Si aucun n'est disponible, conservez la capture de l'erreur et faites installer/activer l'outil prévu par le cours ; ne poursuivez pas les tests OCR avec un exécutable Mac renommé .exe. Ne remplacez pas des erreurs de compilation par des modifications improvisées du code.

Copiez temporairement Cell0.bin, Cell1.bin, CellEmpty.bin du cours dans Execution, puis exécutez successivement :

```
OCR.exe Cell0.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
OCR.exe Cell1.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
OCR.exe CellEmpty.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
```

Résultats attendus : code 0 à chaque fois ; respectivement d: '0' , 97.167969%, d: '1' , 97.753906%, d: '-2' , 0.000000%, chacun suivi d'une fin de ligne. Le programme normal n'écrit rien sur stdout/stderr. Conservez une capture. Les fichiers du cours sont lus, jamais réécrits. Le résultat CellValue.txt est remplacé à chaque appel.

1.2 Tester MATLAB avant de construire le lanceur

Après avoir copié les dernières fonctions .m, effectuez aussi les trois contrôles simples ci-dessous dans MATLAB, avec Current Folder réglé sur Execution :

```
GrilleBinaireValide([0 1; 1 0])
GrilleBinaireValide([0 1; 0 1])
[G, ok] = DirectValues([0 NaN; NaN 0], 2)
```

Attendez successivement TRUE, FALSE, puis G = [0 1; 1 0] avec ok = TRUE. La validation globale ne contient plus de parcours récursif ; les déductions conservent leur parcours équilibré pour respecter le critère « Pas de boucles » sans augmenter la profondeur de récursion. Les commentaires et interfaces du professeur restent inchangés. Ces contrôles sont à exécuter par vous, pas des résultats natifs déjà constatés.

Ouvrez MATLAB vous-même. Placez son **Current Folder** dans Execution, puis copiez ces lignes dans la Command Window :

```
B = [NaN 0 NaN NaN NaN NaN; NaN NaN 0 NaN 0 NaN; NaN NaN NaN NaN 1 0; 1 1 NaN NaN NaN NaN; NaN 0 NaN 0 NaN NaN; NaN NaN NaN NaN 0 NaN];
[S, ok] = SolveBinaire(B)
DisplayBinaire(B, S, 'Binaire_6x6.png', ok)
```

Attendez ok = 1, une grille complète sans modification des indices initiaux et Binaire_6x6.pdf dans Execution. Ouvrez le PDF : indices noirs gras, nouvelles valeurs bleues grasses, tous les traits, nom en haut à gauche et date/heure en haut à droite. La solution peut différer d'un exemple si une grille admet plusieurs solutions ; ses règles et indices doivent toujours être respectés.

Test d'échec logique :

```
B = [0 0 0 NaN; NaN NaN NaN NaN; NaN NaN NaN NaN; NaN NaN NaN NaN];
[S, ok] = SolveBinaire(B)
DisplayBinaire(B, S, 'Contradiction.png', ok)
```

Attendez ok = 0 et un PDF avec == Error ==. Une grille insoluble n'est pas la même chose qu'une panne MATLAB. Une panne doit produire une erreur de lancement/script ; une grille insoluble doit tout de même produire le PDF demandé. Conservez les PDF dans Preuves, puis supprimez les copies de test d'Execution pour éviter de les confondre avec un futur résultat.

2 Les gestes LabVIEW à utiliser partout

2.1 Placer un objet et tirer un fil

1. Ctrl+E alterne entre **Front Panel** (les boutons/indicateurs) et **Block Diagram** (le programme). Ctrl+S enregistre.
2. Sur le diagramme, Ctrl+Space ouvre **Quick Drop**. Tapez le nom exact du nœud anglais indiqué, choisissez le résultat correspondant, appuyez sur Entrée, puis cliquez pour le déposer. N'utilisez pas un Express VI ni un objet IMAQ à la place d'une primitive homonyme.
3. Sur la face avant, clic droit dans le vide ouvre **Controls** ; utilisez sa recherche pour le type indiqué. Une **commande/control** fournit une entrée ; un **indicateur/indicator** affiche une sortie. Clic droit Change to Control/Indicator corrige la direction.
4. Ctrl+H ouvre **Context Help**. Survolez chaque borne avant de câbler : lisez son nom et son type. Quand le guide nomme une borne, trouvez-la par ce nom, pas par sa position supposée sur l'icône.
5. Avec la sélection automatique des outils, approchez une sortie jusqu'à obtenir la bobine, cliquez, puis cliquez sur l'entrée. Pour faire une branche, partez d'un fil existant. Un simple croisement n'est pas une connexion.
6. Donnez à chaque nœud l'identifiant indiqué (N1, E1, etc.) avec une étiquette libre juste à côté. Les identifiants recommencent dans chaque VI. P1 désigne une borne de face avant, pas un nœud à rechercher.
7. Sur un nœud extensible, tirez son bord inférieur pour montrer le nombre exact d'entrées demandé. x,y désignent les deux opérandes affichés dans Context Help. Dans les tableaux, sortie désigne l'unique résultat du nœud ; entrée 0 est la première entrée de haut en bas, même si l'aide la numérote 1.
8. Pour une constante numérique : clic droit Representation puis **I32**, **U32**, **U8** ou **DBL** comme indiqué. N'utilisez pas DBL pour des indices. Créez les constantes d'énumération **depuis la borne** avec Create > Constant, puis choisissez le libellé demandé.
9. Pour obtenir exactement le bon type d'une image, d'un chemin ou d'une erreur : clic droit sur la borne correspondante Create > Control/Indicator/Constant. Ne reconstruisez pas de mémoire les clusters NI.
10. N'utilisez pas Run Continuously. Cliquez une fois sur la flèche **Run** pour un essai. Si la flèche est cassée, cliquez dessus et traitez la première erreur de l'Error List ; gardez une capture si elle ne correspond pas à la recette.

2.2 Tableaux et clusters

- Un tableau 2D est indexé **ligne, colonne**. Array Size renvoie [hauteur, largeur]. En revanche le fichier C commence par **largeur, hauteur**.
- Créez une coquille Array puis déposez son élément dedans. Clic droit sur l'afficheur d'index Add Dimension pour passer de 1D à 2D. Pour un tableau vide, Data Operations > Empty Array ; un tableau [0] n'est pas vide.
- Pour un cluster : placez Cluster, déposez ses éléments, puis clic droit sur la bordure Reorder Controls in Cluster. L'ordre est celui donné par la recette.
- Pour un type partagé .ctl : clic droit sur la commande cluster Advanced > Customize, choisissez **Type Def.**, enregistrez sous le nom prescrit, fermez et acceptez le remplacement. Déposez ensuite ce fichier depuis le projet. N'en recréez pas des copies indépendantes.
- Build Array : **Concatenate Inputs OFF** lorsqu'on assemble des scalaires ; **ON** lorsqu'on concatène des vecteurs en un seul vecteur. Le guide précise les exceptions.

Une **erreur claire** signifie ici : status=FALSE, code=0, source vide. Cela veut dire « aucune erreur ». Un cluster d'erreur n'est pas une chaîne de caractères ; créez-le depuis une borne error in/out.

2.3 Cases et boucles

Tous les sous-VIs sauf ErreurSi.vi ont une **Case Structure de garde**. Câblez le cluster error in à son sélecteur : les cases s'appellent **Error** et **No Error**. Dans Error, pas de lecture, écriture, lancement ou calcul d'index : renvoyez l'erreur inchangée et les sorties neutres indiquées. Dans No Error, placez les nœuds décrits.

Les tunnels de sortie d'une Case doivent être câblés dans **toutes** les cases. Laissez **Use Default If Unwired désactivé**. Une donnée traversant un bord devient un tunnel ; réutilisez ce même tunnel dans l'autre case. Les sorties d'une Case intérieure se raccordent aux sorties de la Case extérieure de même nom, puis aux indicateurs.

Une **garde supplémentaire** après une vérification signifie littéralement : poser une nouvelle Case, brancher l'erreur obtenue au sélecteur, placer les opérations suivantes dans No Error ; dans Error renvoyer les sorties neutres et cette erreur. Ce geste interdit notamment d'indexer un tableau ou de lancer un programme après validation échouée.

Pour chaque For Loop : le terminal i commence à 0. Un tunnel **auto-indexé** distribue un élément/une ligne à chaque tour ; un tunnel **non indexé** transmet l'objet entier. Clic droit sur le tunnel pour choisir. Un **Shift Register** se crée par clic droit sur le bord de la boucle Add Shift Register : câbler sa valeur initiale à **gauche, depuis l'extérieur** ; à l'intérieur lire la borne gauche et écrire la droite à chaque tour. Tous les registres de cette recette sont initialisés. Désactivez le parallélisme des boucles qui écrivent les fichiers ou appellent OCR.

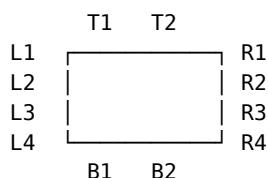
2.4 Chaînes et booléens

Normal Display conserve les caractères tapés tels quels. **Backslash Codes Display** interprète \n comme LF et \r comme CR. Le guide dit explicitement quel affichage utiliser. Un motif regex doit être saisi en **Normal Display**, sans transformer ses barres inverses. Ne recopiez pas les accents typographiques d'un traitement de texte dans les commandes ou formats.

Les commandes booléennes d'options sont des **switches**, pas des boutons à verrouillage/latch. Défaut TRUE pour résoudre et afficher le PDF. Un contrôle numérique affiché arrondi conserve parfois davantage de décimales en mémoire : la recette ne suppose jamais le contraire.

2.5 Connecteurs des sous-VIs

Sur la face avant : clic droit sur l'icône en haut à droite Show Connector, puis Patterns. Choisissez le motif à **12 bornes, 4×2×2×4**. Ne le tournez pas. Convention : L1..L4 = côté gauche de haut en bas ; R1..R4 = côté droit de haut en bas ; T1,T2 = haut gauche à droite ; B1,B2 = bas gauche à droite.



Pour attribuer une borne : cliquez sur elle, puis sur sa commande/son indicateur. Les entrées sont à gauche et les sorties à droite selon les tables. Toutes les bornes non citées restent **non attribuées**. L4 et R4 servent à error in/error out. Pour vérifier, passez la souris avec Context Help sur une instance du sous-VI.

2.6 Sauvegarde et documentation de chaque VI

Après chaque checkpoint : File > VI Properties > Documentation. Copiez la phrase « But » du chapitre, puis ajoutez Auteurs : Louis Pédelaborde, Romeo Mugnier de Almeida, Raphaël Pical. Inscrivez les unités/indices et le comportement en cas d'erreur indiqués. Mettez le diagramme de gauche à droite sans fils passant sous les nœuds. Désactivez Enable automatic error handling dans VI Properties > Execution : les erreurs remontent au VI principal. Revenez à des commandes vides et une erreur claire, puis Edit > Make Current Values Default, Ctrl+S. Les valeurs temporaires de test ne sont pas les valeurs par défaut de remise.

3 Créer les types communs

Construisez les quatre .ctl suivants dans Execution. Tous les éléments sont des **commandes**, dans l'ordre indiqué.

Fichier	Éléments du cluster dans l'ordre	Valeurs par défaut
SeuilsOCR.ctl	vide DBL ; zero DBL ; un DBL	98 ; 90 ; 90
OptionsBinaire.ctl	resoudre booléen ; afficherPDF booléen	TRUE ; TRUE
LignesGrille.ctl	debutX, finX, debutY, finY, chacun tableau 1D I32	quatre tableaux vides
RectangleCellule.ctl	gauche, haut, droite, bas, chacun I32	quatre zéros

Les coordonnées droite/bas sont **exclusives** : une cellule [gauche,droite) a largeur droite-gauche. Les indices commencent à 0. Le cluster Rectangle natif des images NI utilise des I16 : ne le remplacez pas par RectangleCellule.ctl.

Les seuils **98,90,90** servent aux cellules **découpées sans bordures** par cette recette. Le découpage et l'OCR ont été vérifiés séparément sur les PNG6×6,8×8 et4×4 fournis : les cellules vides ont100 % de pixels blancs ; les chiffres environ92,6 à95,6 %. Un seuil vide88 effacerait donc les chiffres. Le réglage **88,90,90 du chapitre1.1 concerne uniquement les trois fichiers binaires d'exemple**, dont le cadrage comporte des bords. Ce ne sont pas des seuils officiels imposés. Pour les chiffres, le C accepte score >= seuil et choisit la plus grande marge score - seuil, conformément à la procédure détaillée de la donnée p18. Le pourcentage écrit reste le score de similitude, pas la marge. Pour la case vide, le test reste strictement > (p10). Les essais LabVIEW sur la VM restent à effectuer.

4 ErreurSi.vi

But : conserver une erreur existante ; sinon créer l'erreur demandée quand Condition est TRUE.

Face avant et connecteur : Condition booléen FALSE L1 ; Code I32 7001 L2 ; Message chaîne vide L3 ; error in L4 ; error out R4. Les codes 6101..6105 et 7001..7012 de la recette sont des codes choisis pour notre programme.

Posez N1 Unbundle By Name champ status, N2 Bundle By Name champs status,code,source, N3 Select, N4 Select, C1 booléen TRUE. Le cluster de base de N2 vient de error in.

Source	Destination
error in	N1.cluster et N2.input cluster
C1 TRUE	N2.status
Code	N2.code
Message	N2.source
Condition	N3.s
N2.cluster	N3.t
error in	N3.f et N4.t
N1.status	N4.s
N3.sortie	N4.f
N4.sortie	error out

Checkpoint : Condition FALSE transmet une erreur claire ; TRUE crée code/message choisis ; si error in contient déjà TRUE/123/test, la sortie reste exactement TRUE/123/test.

5 LireTexte.vi et EcrireTexte.vi

5.1 LireTexte.vi

But : lire tout un fichier texte sans masquer les erreurs et fermer le fichier même si la lecture échoue.

P1 Chemin Path L1 ; P2 error in L4 ; P3 Texte String R1 ; P4 error out R4. Garde extérieure : Error renvoie chaîne vide et P2.

Dans No Error : N1 Open/Create/Replace File, opération **open**, accès **read-only** ; N2 Case sur N1.error out. Branchez P1→N1.file path, P2→N1.error in. Prompt chaîne vide, disable buffering FALSE. Les autres sorties inutilisées restent libres.

N2.Error : Texte vide et N1.error out. N2.No Error : posez une Flat Sequence Structure **deux frames**. Frame 0 : N3 Read from Text File, count I32 **-1**, mode Read Lines OFF et Convert EOL OFF (clic droit).

N1.refnum→N3.file ; N1.error out→N3.error in. N3.text traverse la séquence jusqu'à la sortie Texte. Frame 1 : N4 Close File, N5 Merge Errors à deux entrées. N1.refnum traverse la séquence→N4.refnum ; constante erreur claire FALSE/0/chaîne vide→N4.error in ; N3.error out→N5.entrée0 ; N4.error out→N5.entrée1 ; N5.error out→sortie erreur. Le passage à la frame 1 impose la fermeture **après** la lecture, même si la lecture a échoué.

Reliez sorties Texte/erreur de N2 aux tunnels correspondants de la garde extérieure, puis P3/P4. N2.Error et la garde Error ont bien leurs deux sorties câblées.

5.2 EcrireTexte.vi

But : remplacer/créer le fichier texte, écrire exactement la chaîne reçue, fermer et transmettre les erreurs.

P1 Chemin Path L1 ; P2 Texte String L2 ; P3 error in L4 ; P4 error out R4. Garde extérieure Error : P3→sortie. Dans No Error : même chaîne d'ouverture et de fermeture que ci-dessus, avec les différences **complètes** suivantes :

1. N1.operation = **replace or create**, access = **write-only** ; P3→N1.error in ; P1→file path.
2. N2.Error ne renvoie que N1.error out.
3. N2.No Error, frame 0 : N3 est **Write to Text File**, Convert EOL OFF ; N1.refnum→file ; P2→text ; N1.error out→error in. Aucun LF n'est ajouté automatiquement.
4. Frame 1 : N4 Close File avec erreur claire ; N5 Merge Errors reçoit N3.error out puis N4.error out, dans cet ordre ; résultat→N2→garde→P4. Refnum N1 traverse le tunnel de séquence.

Checkpoint : écrivez une chaîne abc\n saisie en Backslash Codes Display dans un fichier de test, relisez-la avec LireTexte : String Length doit donner 4. Recommencez avec x\n : longueur 2, pas de reste c de l'ancien fichier. Un dossier à la place du fichier doit produire une erreur, pas être remplacé.

6 SupprimerResultat.vi

But : effacer un ancien fichier généré avant une nouvelle exécution ; ne jamais supprimer un dossier.

P1 Chemin Path L1 ; P2 error in L4 ; P3 error out R4. Garde Error→P2. No Error : N1 Check if File or Folder Exists, N2 Case sélectionnée par file or folder exists?. P1→N1.path, P2→N1.error in. N2.FALSE : P2→sortie. N2.TRUE : N3 File/Directory Info, N4 ErreurSi.vi, N5 Case d'erreur, N6 Delete dans N5.No Error.

Le booléen de N1 impose l'ordre de cette Case ; aucune sortie error out de N1 n'est nécessaire. Câblez P1→N3.path ; P2→N3.error in ; N3.directory→N4.Condition ; I32 7002→N4.Code ; chaîne Le chemin du resultat est un dossier.→N4.Message ; N3.error out→N4.error in ; N4.error out→N5.sélecteur. N5.Error transmet N4.error out. N5.No Error : P1→N6.path ; N4.error out→N6.error in ; FALSE→entire hierarchy ; FALSE→confirm ; chaîne vide→prompt ; N6.error out→sortie. Toutes les sorties erreur remontent N5→N2→garde→P3.

N'utilisez cette fonction que pour les trois chemins générés par le programme : CellValue.txt, solve.m, et le PDF attendu. Jamais pour le PNG, les sources ou un dossier choisi arbitrairement. Checkpoint : fichier absent = succès ; fichier présent = supprimé ; dossier présent = erreur 7002 et dossier intact.

7 LireImage.vi

But : lire le PNG avec les fonctions natives LabVIEW, afficher l'image et fournir un tableau 2D TRUE=noir, FALSE=blanc. Aucune lecture d'image n'est déléguée à C, MATLAB ou Python.

7.1 Face avant et objets

P1 PNG Path L1 ; P2 error in L4 ; P3 Bitmap Boolean 2D R1 ; P4 Image **2D Picture** R2 ; P5 Largeur I32 R3 ; P6 Hauteur I32 T2 ; P7 error out R4. Créez Image depuis la sortie new picture de Draw Flattened Pixmap. Fond de l'indicateur blanc : View > Tools Palette, outil couleur, clic droit dans la zone de dessin, blanc.

La garde Error renvoie Bitmap vide 0×0, picture vide, largeur/hauteur 0, P2. Dans No Error, construisez les étapes suivantes dans cet ordre. Une garde après chaque validation empêche de convertir/indexer après erreur. Chacune de ses branches Error renvoie les mêmes quatre valeurs neutres et l'erreur de cette étape.

7.2 Lecture et dimensions

- N1 Read PNG File.vi : P1→path to PNG file, P2→error in, constante U8 128→Transparency Thresh. N1.error out sélectionne la garde G1 ; son image data entre dans G1.No Error. La sortie path de N1 est inutilisée.
- Dans G1.No Error : U1 Unbundle By Name de image data, champ **Rectangle**. U2 Unbundle By Name du Rectangle, champs left/top/right/bottom. Posez quatre To Long Integer T1..T4 ; left→T1, top→T2, right→T3, bottom→T4.
- N2 Subtract : T3–T1=Largeur. N3 Subtract : T4–T2=Hauteur. Faites six comparaisons : Largeur<32, Largeur>4096, Hauteur<32, Hauteur>4096, T1≠0, T2≠0. Chaque comparaison est un nœud séparé et a ses deux entrées câblées avec la constante I32 concernée.
- Posez Build Array à six entrées scalaires booléennes, **Concatenate Inputs OFF**, puis Or Array Elements. Câblez les six résultats dans l'ordre ci-dessus→Build Array→OR. OR→ErreurSi.Condition ; code 6101 ; message Dimensions PNG invalides. ; N1.error out→error in. L'erreur obtenue sélectionne G2. Largeur, Hauteur, Rectangle original et image data traversent dans G2.No Error.

32..4096 est une borne de travail explicite ; l'image n'a pas besoin d'être carrée en pixels. La grille en cellules doit, elle, être carrée. Une image Word renommée .png doit échouer dès la lecture, pas être prise pour une image vide.

7.3 Conversion native en un bit

Dans G2.No Error, posez D1 Draw Flattened Pixmap.vi, D2 Picture to Pixmap.vi, D3 Unflatten Pixmap.vi, A1 Array Size, I1/I2 Index Array 1D.

Source	Destination
Picture vide créé depuis D1.picture	D1.picture
N1.image data	D1.image data
D1.new picture	D2.picture et tunnel Image final
Rectangle original U1	D2.rect
I32 1	D2.depth
U32 16777215, blanc RGB	D2.Background Color
D2.image data	D3.image data

Source	Destination
D3.1-bit pixmap	A1.array et tunnel brut
A1.sizes	I1.array et I2.array
I32 0	I1.index
I32 1	I2.index

N'employez aucune autre sortie pixmap de D3. D1/D2/D3 n'ont pas à recevoir des bornes d'erreur inventées. D3.colors est conservé pour la polarité. Les autres sorties éventuelles de D2, ainsi que D3.top left/mask/autres profondeurs, sont inutilisées.

Vérifiez I1=Hauteur et I2=Largeur avec deux Not Equal? puis OR. Dépliez D2.image data avec Unbundle By Name, champ image depth : comparer à 1, ajouter ce résultat à OR. OR→ErreurSi.Condition, code 6101, message Conversion image incohérente., erreur G2→error in. Garde G3 sur son résultat. G3.No Error reçoit brut, colors, Image, Largeur, Hauteur.

7.4 Normaliser la polarité

D3 renvoie des indices de palette : TRUE n'est pas à supposer noir sans vérification. Posez Array Size sur colors puis une Case numérique sur sa longueur, avec les cases **2, 0, Default**. Ses sorties sont noirVautTrue Boolean et erreur.

Case 2 : I1/I2 Index Array, indices I32 0 et 1 sur colors. Pour chaque couleur U32, placez Color to RGB ; branchez color0/color1 à leur entrée couleur. Convertissez chacun des six résultats R/G/B en U32 avec To Unsigned Long Integer avant addition. Pour color0, deux Add calculent $(R0+G0)+B0$; pour color1, deux Add calculent $(R1+G1)+B1$. Less? somme1<somme0→noirVautTrue. Equal? somme1=somme0→ErreurSi.Condition ; code 6105 ; message Palette sans contraste. ; erreur G3→error in ; résultat→erreur de Case.

Case 0 : placez deux constantes booléennes sur le diagramme, étiquetées PolariteVerifiee FALSE et NoirVautTrue TRUE. NOT(PolariteVerifiee)→ErreurSi.Condition ; code 6105 ; message Verifier la polarite avec Binairo_6x6.png avant execution. ; erreur G3→error in. NoirVautTrue→sortie booléenne ; ErreurSi.error out→sortie erreur. Ce réglage sera effectué une seule fois au checkpoint ci-dessous si la palette est vide.

Default : FALSE→noirVautTrue ; ErreurSi(TRUE,6105,Palette 1 bit invalide.,erreur G3)→sortie erreur.

Après cette Case : NOT sur le tableau brut entier ; Select avec s=noirVautTrue, t=brut, f=NOT(brut). La sortie de Select est le bitmap normalisé. Garde finale G4 sur l'erreur de palette : No Error renvoie ce Bitmap, Image, Largeur, Hauteur et erreur ; Error renvoie les valeurs neutres. Remontez ces cinq sorties dans G3→G2→G1→garde extérieure→indicateurs. Dans chaque branche Error extérieure, le cluster est celui qui a déclenché la branche, pas une erreur claire.

7.5 Checkpoint image et palette

- Ouvrez LireImage seul, choisissez le vrai Binairo_6x6.png, lancez une fois. Si l'erreur est 6105 et demande de vérifier la polarité, mettez une sonde sur le fil brut, pas sur Bitmap neutralisé après erreur.
- Dans le tableau de la sonde, regardez [ligne0,colonne0] et [20,20]. Dans ce fichier précis, le premier est noir, le second blanc.
- Brut TRUE puis FALSE : NoirVautTrue=TRUE. Brut FALSE puis TRUE : NoirVautTrue=FALSE. Dans ces deux cas seulement, mettez PolariteVerifiee=TRUE, sauvegardez et relancez. Deux valeurs identiques : ne validez pas ; capturez le PNG choisi, la sonde et l'erreur pour correction.
- Attendez largeur 454, hauteur 431, Bitmap[0,0]=TRUE, Bitmap[20,20]=FALSE et aucune erreur. La grille doit être visible sans inversion des noirs/blancs.

5. Avec Binairo_8x8.png, attendez largeur 610, hauteur 577 ; Bitmap[5,2]=TRUE et Bitmap[30,30]=FALSE. Les tailles sont celles des images reçues ; si Moodle a remplacé les fichiers, ne forcez pas ces nombres dans le diagramme.
6. Déplacez l'image dans un autre dossier, re-sélectionnez-la : elle doit encore être lue. La calibration n'est à refaire que si la version/environnement de conversion change.

8 Géométrie de la grille

Construisez maintenant, dans cet ordre : `EtendueNoire.vi`, `IntervallesNoirs.vi`, `ComputeCellRect.vi`, `ComputeRowsCols.vi`, en suivant l'**annexe G — Détecter la grille et découper ses cases** placée à la fin de cette recette. Revenez ensuite au chapitre 9. Les bornes de cellule sont **10 à 100 pixels**, identiques à celles du C actuel.

9 EcrireCellule.vi

But : écrire exactement le format attendu par LireCellule en C, sans dimensions supplémentaires et sans inverser largeur et hauteur.

P1 Cellule Boolean 2D L1 ; P2 Chemin Path L2 ; P3 error in L4 ; P4 error out R4. Garde Error transmet P3.

9.1 Valider puis convertir

Dans No Error : A1 Array Size sur P1 ; I1/I2 Index Array sur A1, indices 0/1 ; I1=hauteur, I2=largeur. Deux In Range and Coerce reçoivent hauteur/largeur avec lower=10 et upper=100, **Include lower limit et Include upper limit TRUE**. AND des deux in range?, puis NOT→ErreurSi.Condition ; code 7003 ; message Cellule hors des bornes 10 a 100 pixels. ; P3→error in. Garde G1 sur le résultat.

G1.No Error :

1. H1/H2 To Unsigned Long Integer sur largeur puis hauteur. H3 Build Array deux scalaires, Concatenate OFF : H1→entrée0, H2→entrée1. Résultat = **U32 [largeur,hauteur]**.
2. F1 For Loop : P1 Cellule→tunnel auto-indexé, qui fournit une ligne Boolean 1D ; N non câblé. Dans F1, F2 For Loop : ligne→tunnel auto-indexé, qui fournit un booléen ; N non câblé. Dans F2, Select : s=pixel, t=U8 1, f=U8 0. Sortie auto-indexée de F2 = ligne U8. Cette ligne sort par tunnel auto-indexé de F1 = matrice U8.
3. M1 Multiply largeur×hauteur en I32. M2 Reshape Array sur la matrice U8, **une seule dimension de sortie**, M1→dimension size. Résultat = vecteur U8 dans l'ordre ligne par ligne. Ne transposez rien.

9.2 Écrire et fermer

N1 Open/Create/Replace File : operation **replace or create**, access **write-only**, disable buffering FALSE, prompt vide ; P2→file path ; erreur G1→error in. Garde G2 sur N1.error out. Error transmet cette erreur. No Error contient une Flat Sequence de deux frames.

Frame 0 : N2/N3 Write to Binary File. N1.refnum→N2.file ; H3→N2.data ; FALSE→N2.prepend array or string size? ; constante créée depuis byte order réglée **2 little-endian**→N2.byte order ; N1.error out→N2.error in. N2.refnum out→N3.file ; M2→N3.data ; FALSE→N3.prepend ; little-endian→N3.byte order ; N2.error out→N3.error in. Prompt vide pour les deux. Les sorties cancelled restent libres.

Frame 1 : N4 Close File reçoit refnum N1 et erreur claire ; N5 Merge Errors reçoit N3.error out puis N4.error out. Résultat→G2→G1→garde extérieure→P4. La séquence impose que les deux écritures finissent avant la fermeture. **Ne branchez pas le chemin à la seconde écriture : cela pourrait rouvrir/remplacer le fichier au lieu de continuer après l'en-tête.**

Le fichier contient exactement $8 + \text{largeur} \times \text{hauteur}$ octets. Le prepend=FALSE est volontaire : nous avons écrit nous-mêmes l'en-tête U32 dans l'ordre du C. Le raccourci consistant à préfixer directement une matrice LabVIEW risquerait d'écrire hauteur puis largeur.

9.3 Checkpoint asymétrique sans outil externe

Créez un VI temporaire ControleBinaire.vi dans un sous-dossier Binaire/Preuves/Essais, pas dans les fichiers finaux à remettre.

1. Initialize Array : élément FALSE, dimension0=35, dimension1=40. Replace Array Subset : cette matrice→array, row=2, col=9, new element=TRUE.
2. Résultat→EcrireCellule.Cellule ; créez une commande Path pour un fichier temporaire Asymetrique.bin, erreur claire→error in, affichez error out.

3. Après son error out, posez Read from Binary File avec le **chemin** du fichier, constante de type U8 scalaire→data type, count I32 -1, byte order little-endian. La lecture U8 brute renvoie tous les octets ; ne lui donnez pas un type tableau qui attendrait une taille préfixée. Créez un indicateur tableau sur la sortie et un Array Size→indicateur.
4. Exécutez : taille 1408 ; huit premiers octets décimaux [40,0,0,0,35,0,0,0]. À l'index **97**, valeur 1 ; les autres pixels 0. L'index 97 est $8 + 2 \times 40 + 9$.
5. Testez ensuite une matrice 9×40 : erreur 7003. Revenez aux valeurs normales. Ce contrôle est de développement ; gardez-le dans Preuves, pas dans le code évalué si seuls les VIs de fonctionnement sont demandés.

10 CommandeOCR.vi

But : construire l'appel OCR avec le **chemin complet de Cell.bin**, comme demandé par la donnée p23, et les trois seuils. Les chemins sont calculés depuis le dossier du VI, jamais inscrits comme constantes personnelles.

P1 Seuils SeuilsOCR.ctf L1 ; P2 error in L4 ; P3 Commande String R1 ; P4 error out R4 ; **P5** Dossier **Path L2**, dossier absolu d'Execution. Garde Error : chaîne vide/P2.

Dans No Error : U1 Unbundle By Name vide/zero/un. Trois In Range and Coerce R1/R2/R3, bornes DBL0 et100 incluses ; vide→R1.x, zero→R2.x, un→R3.x. Leurs trois in range?→Build Array→And Array Elements→NOT→ErreurSi.Condition ; Code7004 ; Message Les trois seuils doivent etre compris entre 0 et 100. ; P2→error in. Garde G1 sur le résultat. NaN et Inf sont refusés ; ne pas utiliser les sorties coerced x pour masquer une entrée invalide.

Dans G1.No Error, placer et câbler :

Objet	Entrées	Sortie
B1 Build Path	P5→base path ; chaîne OCR.exe→name	chemin absolu OCR
B2 Build Path	P5→base path ; chaîne Cell.bin→name	chemin absolu cellule
S1 Path To String	B1.path	chaîne exécutable
S2 Path To String	B2.path	chaîne cellule
F1 Format Into String	format ci-dessous ; initial string vide ; erreur G1→error in	commande et erreur

F1 reçoit **cinq arguments**, dans cet ordre : S1, S2, U1.vide, U1.zero, U1.un. Saisir le format en **Normal Display** :

```
%.;"%s" "%s" %.15f %.15f %.15f
```

F1.resulting string→Commande ; F1.error out→sortie erreur. G1.Error : chaîne vide et erreur G1. Remonter les deux tunnels vers la garde extérieure puis P3/P4.

System Exec lancera **l'exécutable directement** : ne pas ajouter cmd /c à cette commande. Les guillemets protègent les chemins contenant des espaces ; aucun shell n'a à développer un nom de dossier. P5 provient de Current VI's Path→Strip Path dans le principal, puis est transmis par LireCase. Working directory reste câblé séparément dans System Exec au même dossier.

Checkpoint : pour un dossier de test choisi C:\Travail Binairo\Execution, seuils98/90/90, obtenir exactement :

```
"C:\Travail Binairo\Execution\OCR.exe" "C:\Travail Binairo\Execution\Cell.bin" 98.00000000000000 90.00000000000000 90.00000000000000
```

Ce chemin est un exemple de résultat : ne le copiez pas dans le diagramme. %. ; force le point décimal et disparaît de la commande produite. Le format conserve quinze décimales sans exposant. Testez -1,101,NaN : erreur et commande vide. Le checkpoint de LireCase et V19 vérifieront l'appel réel sous Windows, y compris dans un dossier avec espaces.

11 AnalyserResultatOCR.vi

But : transformer une réponse valide de C en 0, 1 ou NaN pour MATLAB. Toute autre réponse est une erreur.

P1 Texte String L1 ; P2 error in L4 ; P3 ValeurMatlab String R1 ; P4 Pourcentage DBL R2 ; P5 error out R4.
Sorties neutres : chaîne vide, DBL0, erreur transmise.

11.1 Vérifier toute la ligne

Dans la garde No Error : N1 Match Regular Expression ; étirez pour montrer **deux submatches**. Branchez P1→string ; I32 0→offset ; FALSE→multiline? et ignore case? ; P2→error in. Constante regex **Normal Display :**

```
^d:'(-2|0|1)', ([0-9]{1,3}\.[0-9]{6})%\r?\n$
```

N2 String Length reçoit P1. N3 Not Equal? compare N1.offset past match à N2.length. N3→ErreurSi.Condition ; code7005 ; message Format de CellValue.txt invalide. ; N1.error out→error in. Garde G1 après ErreurSi. N1.submatch1 contient le symbole et N1.submatch2 le score ; le whole match n'est aucun de ces deux champs.

G1.No Error : N4 Fract/Exp String To Number : submatch2→string, offset I32 0, default DBL0, **use system decimal point? FALSE**. N5 In Range and Coerce sur son nombre, bornes0/100 incluses ; NOT(in range?)→ErreurSi code7005 message Pourcentage OCR invalide.. Erreur G1→error in. Garde G2 après ce contrôle.

G2.No Error : Case **sur la chaîne submatch1**, avec quatre cases :

Case	ValeurMatlab	Pourcentage	Erreur
0	constante chaîne 0	N4.number	erreur G2
1	constante chaîne 1	N4.number	erreur G2
-2	constante chaîne NaN	N4.number	ErreurSi(N4.number≠0,7005,Score de case vide invalide.,erreur G2)
Default	chaîne vide	DBL0	ErreurSi(TRUE,7005,Symbole OCR invalide.,erreur G2)

Pour la case -2, ajoutez une garde sur cette dernière erreur : Error→valeurs neutres, No Error→NaN/score/erreur. Remontez les trois tunnels dans G2→G1→garde extérieure→P3/P4/P5. Toutes les branches Error intermédiaires utilisent les valeurs neutres.

11.2 Checkpoint du parseur

Saisissez les entrées ci-dessous en **Backslash Codes Display** pour fabriquer les fins de ligne, puis repassez en Normal Display si désiré.

Texte d'essai	Résultat
d: '-2', 0.000000%\n	NaN,0,sans erreur
d: '0', 97.167969%\n	0,97.167969,sans erreur
d: '1', 97.753906%\r\n	1,97.753906,sans erreur
d: '2', 90.000000%\n	erreur7005
d: '0', 101.000000%\n	erreur7005

Texte d'essai	Résultat
d: '0', 90,000000%\n	erreur7005
d: '0', 90.000000% sans LF	erreur7005
ligne correcte puis seconde ligne	erreur7005
chaîne vide	erreur7005

12 LireCase.vi

But : traiter exactement une cellule, dans l'ordre découpage → fichier → OCR → validation → lecture du résultat. Une erreur ne devient jamais une case vide.

12.1 Face avant

Commande ou indicateur	Type	Connecteur
Bitmap	Boolean2D commande	L1
Lignes	LignesGrille.ctl commande	L2
Indices	cluster commande ligne l32 puis colonne l32	L3
error in	erreur commande	L4
Dossier	Path commande	T1
Seuils	SeuilsOCR.ctl commande	T2
ValeurMatlab	String indicateur	R1
Pourcentage	DBL indicateur	R2
Diagnostic	String indicateur	R3
error out	erreur indicateur	R4

Garde Error : chaîne vide,DBL0,chaîne vide,error in. No Error contient U1 Unbundle By Name ligne/colonne, V1 ComputeCellRect.vi, U2 Unbundle By Name du Rectangle, deux Subtract et V2 Array Subset 2D.

12.2 Découper et écrire

- Indices→U1.cluster ; U1.ligne/colonne→V1.ligne/colonne ; Lignes→V1.Lignes ; error in→V1.error in. V1.Rectangle→U2 ; U2.droite–gauche→largeur ; U2.bas–haut→hauteur.
- Bitmap→V2.array ; U2.haut→V2.index0 ; hauteur→V2.length0 ; U2.gauche→V2.index1 ; largeur→V2.length1. Garde G1 sur V1.error out **avant toute écriture**. Les calculs purs peuvent être à gauche de G1, mais V2 peut aussi être placé dedans.
- Dans G1.No Error : B1/B2 Build Path : Dossier→base path des deux ; chaîne Cell.bin→B1.name ; chaîne CellValue.txt→B2.name.
- V3 SupprimerResultat.vi : B2→Chemin, V1.error out→error in. V4 EcrireCellule.vi : V2.subarray→Cellule, B1→Chemin, V3.error out→error in.
- V5 CommandeOCR.vi : Seuils→Seuils, **Dossier→Dossier**, V4.error out→error in. V6 System Exec.vi : V5.Commande→command line, **Dossier→working directory**, TRUE→wait until completion?, TRUE→run minimized?, chaîne vide→standard input, constante créée depuis expected output size réglée 65536, V5.error out→error in.

12.3 Vérifier le programme avant de lire son fichier

Posez C1 Not Equal? return code≠l32 0 ; C2 String Length sur standard error ; C3 Greater? longueur>0 ; C4 OR(C1,C3). Posez F1 Format Into String avec **quatre arguments** ligne,colonne,return code,standard error, format saisi Backslash Codes Display :

```
Cellule ligne %d colonne %d : code OCR %d\nstderr : %s\n
```


Indices.ligne→arg0, colonne→arg1, V6.return code→arg2, V6.standard error→arg3 ; erreur claire→F1.error in ; initial string vide. F2 Concatenate Strings : F1.result, chaîne stdout : , V6.standard output ; résultat→Diagnostic et Message du V7 ErreurSi. C4→V7.Condition ; 7006→Code. Merge Errors à deux entrées reçoit V6.error out puis F1.error out→V7.error in.

V8 LireTexte.vi : B2→Chemin ; V7.error out→error in. V9 AnalyserResultatOCR.vi : V8.Texte→Texte ; V8.error out→error in. V9.ValeurMatlab/Pourcentage/error out→sorties correspondantes ; Diagnostic reste celui de F2 même en cas d'échec. Remontez G1 puis la garde extérieure vers les quatre indicateurs. G1.Error renvoie neutres et V1.error out.

Pourquoi supprimer le résultat avant chaque cellule : un ancien fichier ne doit pas survivre à un exécutable manquant ou défectueux. Le code de retour et le cluster System Exec sont deux contrôles distincts. Notre enveloppe LabVIEW considère stderr non vide comme diagnostic d'échec ; stdout est conservé pour inspection.

Checkpoint : pour la première cellule du 6×6, indices0,0, les coordonnées sont gauche6, haut6, droite78, bas74, soit **72×68** et Cell.bin de **4904 octets**. Résultat NaN. La case0,1 doit donner 0. Les fichiers Cell0.bin/Cell1.bin du cours sont d'autres découpages avec bords : leurs dimensions74×74 ne sont pas le résultat attendu ici.

13 GenererScript.vi

But : produire solve.m depuis la matrice réellement reconnue par les boucles LabVIEW. Aucun exemple prérempli ne remplace cette génération.

P1 MatriceTexte String L1 ; P2 PNG Path L2 ; P3 Options OptionsBinaire.ctl L3 ; P4 error in L4 ; P5 Dossier Path T1 ; P6 Script String R1 ; P7 CheminScript Path R2 ; P8 error out R4. Garde Error : chaîne vide, chemin vide, P4.

Dans No Error, posez les objets suivants et câblez exactement :

Objet	Entrées	Sortie utilisée
N1 Strip Path	P2→path	name, nom PNG seulement
N2 Search and Replace String	N1.name→string ; ' '→search ; ' '→replace ; I32 0→offset ; TRUE→replace all? ; mode regex OFF	chaîne avec apostrophes doublées
N3 Unbundle By Name	P3→cluster ; champs resoudre,afficherPDF	deux booléens
N4 Select	s=N3.resoudre ; t=chaîne true ; f=chaîne false	option résolution
N5 Select	s=N3.afficherPDF ; t=true ; f=false	option PDF
N6 Format Into String	format ci-dessous ; arg0=P1 ; arg1=N2 ; arg2=N4 ; arg3=N5 ; initial string vide ; P4→error in	texte script et erreur
N7 Build Path	base=P5 ; name=solve.m	chemin script
N8 EcrireTexte.vi	Chemin=N7 ; Texte=N6 ; error in=N6.error out	erreur finale

Le format N6 se saisit en **Normal Display**, avec de vrais retours à la ligne. Agrandissez la constante String et copiez tout le bloc suivant. Ne tapez pas les deux caractères \n à la place des retours à la ligne. Le seul \n écrit littéralement ci-dessous est celui des chaînes MATLAB de fprintf :

```
B = [  
%s];  
fichierSource = '%s';  
printGrid(B);  
if %s  
    [S, ok] = SolveBinaire(B);  
else  
    S = B;  
    ok = GrilleBinaireValide(B);  
end  
DisplayBinaire(B, S, fichierSource, ok);  
if %s  
    uiopen(CheminPdfBinaire(fichierSource), 1);  
end  
if ok  
    fprintf('Binaire traite.\n');  
else  
    fprintf('Grille contradictoire ou sans solution.\n');  
end
```

N6.result→P6 ; N7.path→P7 ; N8.error out→P8, via la garde. Les %s ne sont pas des instructions MATLAB : ils sont remplacés par Format Into String. Les options deviennent **true/false**, jamais les nombres 0/1, pour produire des conditions MATLAB explicites. Les appels à SolveBinaire et DisplayBinaire figurent directement dans solve.m.

Checkpoint manuel : MatriceTexte contient deux lignes 0 1;\n1 0;\n, PNG choisi Grille.png, options TRUE/FALSE. Ouvrez le fichier enregistré avec un éditeur de texte :

```

B = [
0 1;
1 0;
];
fichierSource = 'Grille.png';
printGrid(B);
if true
    [S, ok] = SolveBinaire(B);
else
    S = B;
    ok = GrilleBinaireValide(B);
end
DisplayBinaire(B, S, fichierSource, ok);
if false
    uiopen(CheminPdfBinaire(fichierSource), 1);
end
if ok
    fprintf('Binaire traite.\n');
else
    fprintf('Grille contradictoire ou sans solution.\n');
end

```

Le nom affiché dans le PDF est le nom du PNG ; il n'est pas nécessaire de transmettre son chemin complet à MATLAB, qui n'a pas à relire l'image. Un nom L'exemple.png doit devenir 'L' 'exemple.png'. Les noms fournis dans les essais du cours sont ASCII ; validez séparément l'encodage natif si vous employez des noms accentués sur Windows. Ne créez pas de protocole MatlabStatus.txt : les sources actuelles n'en écrivent pas.

14 Option facultative : inspecter le lanceur fourni

La donnée exige de lancer MATLAB avec le script généré et de détecter ses erreurs (p26 et36). **Elle n'impose pas MP_LaunchMatlabScript4.vi**. Ce VI est proposé dans les fichiers utiles et employé par l'exercice. Le mode direct du chapitre15.2 est donc le parcours normal de cette recette ; vous pouvez passer directement à ce chapitre sans inspecter le lanceur fourni. La page6 de l'exercice LabVIEW2 indique son rôle, sa co-localisation avec le script et une option interne de test ; elle ne donne pas les noms/types de toutes ses bornes. Cette inspection ne peut pas être remplacée par un connecteur inventé.

1. Ouvrez la copie d'Execution dans LabVIEW, sans modifier l'original Moodle. Montrez son Front Panel et son connector pane. Activez Context Help.
2. Cliquez chaque borne connectée du connector pane : la commande/indicateur correspondant doit se mettre en évidence. Notez dans Preuves/lanceur.txt pour **chaque** borne : position, nom exact, commande ou indicateur, type, valeur par défaut. Pour un numérique, clic droit Representation. Pour un cluster, notez ses éléments.
3. Ctrl+E : repérez l'option interne qui choisit un script de test. Vérifiez que la branche de fonctionnement utilise l'entrée de script. Ne laissez pas le test interne sélectionné.
4. Suivez le fil du nom/script : le VI attend-il solve, solve.m, ou un chemin complet ? Vérifiez par les nœuds de construction du chemin et par un essai manuel avec un petit script. Ne concluez pas à partir du seul libellé.
5. Repérez System Exec.vi ou le mécanisme de lancement. Son wait until completion? doit permettre de savoir quand le script est fini. Sous Windows, si MATLAB est lancé en ligne de commande, vérifiez le comportement synchrone effectif. Le simple fait que le VI retourne n'est pas une preuve.
6. Relevez la sortie error out, et les sorties éventuelles return code/stdout/stderr. Identifiez ce que le VI fait si MATLAB manque ou si le script appelle error('Essai volontaire');. Gardez les deux captures, succès et erreur.
7. Si ces essais donnent une erreur fiable et une attente complète, câblez le mode fourni du chapitre15. S'il ne fournit pas ces garanties, **ne mettez pas un Wait de durée arbitraire** et ne marquez pas le lanceur comme validé. Le mode direct du chapitre15 permet de tester le reste immédiatement ; gardez le mode direct pour la remise ; ne demandez une adaptation que si vous souhaitez cette option.
8. Si une nouvelle version du lanceur ou un Binairo...ConPane.vi est disponible sur Moodle, conservez aussi le fichier et son aide. Le connecteur de Billard2025_ConPane.vi ne doit pas être copié dans ce projet.

L'interface inconnue de ce VI facultatif ne bloque ni la construction ni la validation du mode direct. N'ajoutez pas une dépendance au VI fourni si vous ne l'utilisez pas.

15 LancerMatlab.vi

But : lancer `solve.m`, attendre sa fin, conserver le diagnostic et transmettre une erreur réelle en cas d'échec. Il n'effectue pas le calcul du Binaire lui-même.

15.1 Interface et préparation

P1 Dossier Path L1 ; P2 ExecutableMatlab Path L2 ; P3 LanceurFourni booléen FALSE L3 ; P4 error in L4 ; P5 Diagnostic String R1 ; P6 error out R4. **FALSE est la valeur normale et le défaut enregistré.** TRUE est réservé à l'option du lanceur fourni si vous la construisez et la validez.

Garde Error : Diagnostic vide et P4. No Error : N1 Build Path base=P1, name=solve.m ; Case C1 sélectionnée par P3. C1.FALSE est le mode direct suivant ; C1.TRUE est le mode fourni décrit ensuite. Les deux renvoient Diagnostic et erreur aux sorties de la garde.

15.2 Mode direct entièrement déterminé

1. Dans C1.FALSE, posez N2 Path To String sur P2. Vérifiez le chemin avec N3 Match Regular Expression, offset0,multilineFALSE,ignorecaseFALSE,error in=P4, regex **Normal Display** :

```
^[A-Za-z0-9_\.:\\\-]+[.][eE][xX][eE]$
```

2. N4 String Length(N2) ; N5 Not Equal?(N3.offset past match,N4.length)→ErreurSi.Condition, code7008, message Choisir matlab.exe ; ce mode accepte un chemin ASCII sans caracteres de commande., error in=N3.error out. Garde G1. Cette restriction concerne **la commande shell du mode direct**, pas le chemin PNG lu nativement. Le chemin d'installation courant Program Files est accepté. Un chemin valide contenant d'autres caractères doit passer par l'adaptation du lanceur fourni, pas être modifié au hasard.

3. Dans G1.No Error, N6 Format Into String : initial string vide, error in=erreur G1, **un argument N2.** Format Normal Display :

```
cmd /d /s /c "%s" -wait -batch "run('solve.m')"
```

4. N7 System Exec : N6.result→command line ; P1→working directory ; TRUE→wait until completion? ; TRUE→run minimized? ; chaîne vide→standard input ; constante expected output size=262144 créée depuis sa borne ; N6.error out→error in.
5. N8 Format Into String trois arguments N7.return code, N7.standard output, N7.standard error ; erreur claire→error in, initial string vide. Format Backslash Codes Display : MATLAB code %d\nstdout : \n%s\nstderr : \n%s\n.
6. N9 Not Equal?(N7.return code,0)→N10 ErreurSi.Condition ; code7008 ; N8.result→Message et Diagnostic. Merge Errors(N7.error out,N8.error out)→N10.error in ; N10.error out→sortie erreur. G1.Error renvoie diagnostic vide et erreur G1.

MATLAB peut écrire un avertissement dans stderr sans échec : on conserve ce texte, mais le code de retour et le cluster d'erreur déterminent l'échec. L'option -batch doit être disponible dans MATLAB (R2019a ou plus récent). Vérifiez dans la VM ; si elle est plus ancienne, utilisez le lanceur du cours et retournez sa version au lieu d'inventer une commande différente.

15.3 Mode fourni et adaptation exacte à l'interface observée

Parcours normal, sans lanceur fourni : dans C1.TRUE, placez ErreurSi(TRUE,7009,option lanceur fourni non installée ; utiliser le mode direct.,P4) ; câblez son error out à la sortie erreur et la même chaîne à Diagnostic. La case FALSE contient le lancement complet et constitue le fonctionnement normal. Cette branche TRUE signale une option indisponible, pas une fonction requise manquante. Gardez LanceurFourni=FALSE par défaut.

Seulement si vous installez l'option : remplacez ces deux sorties par l'intégration suivante et placez **le vrai MP_LaunchMatlabScript4.vi**. Utilisez la fiche du chapitre14 pour appliquer cette table, uniquement aux bornes qui existent réellement :

Borne observée	Câblage
Entrée script de type Path	N1.path→cette borne
Entrée script String dont le diagramme attend le chemin complet	Path To String(N1.path)→cette borne
Entrée script String attendant le nom avec extension	constante solve.m→cette borne
Entrée script String attendant le nom sans extension	constante solve→cette borne
Entrée dossier de travail Path, si présente	P1→cette borne
Entrée exécutable Path, si présente	P2→cette borne
Entrée exécutable String, si présente	Path To String(P2)→cette borne
error in, si présente	P4→cette borne
error out, si présente et testée	cette borne→chaîne de contrôle d'erreur
Sélection de test interne exposée, si présente	valeur choisissant le script fourni, vérifiée au chapitre14

Toutes les autres entrées doivent être identifiées et leur défaut confirmé par l'aide/diagramme : **une borne inconnue est une information à retourner, pas une invitation à deviner**. Si le VI n'a pas error in, il reste protégé par la garde extérieure de LancerMatlab. Si error out n'existe pas ou ne reflète pas une vraie erreur de script, ce mode n'est pas encore validé : conservez un ErreurSi(TRUE,7009,Interface du lanceur fourni a verifier.,P4) et diagnostic identique jusqu'à correction fondée sur sa vraie interface. Cela empêche un faux succès.

Si return code/stdout/stderr existent, reprenez N8/N9/N10 du mode direct avec ces **sorties réelles**. S'il n'existe qu'un error out fiable, celui-ci devient la sortie erreur ; Diagnostic vaut une chaîne constante Lanceur fourni termine ; verifier le PDF et error out.. Le contrôle du fichier frais, au chapitre16, reste obligatoire. Il ne remplace jamais une sortie d'erreur fiable.

Checkpoint pour les deux modes validés : script 2×2 de chapitre13→fin d'exécution, PDF nouveau ; script error('Essai volontaire') ;→erreur visible et aucun succès annoncé. Renommez temporairement l'exécutable MATLAB sélectionné **dans la commande de test en choisissant un chemin inexistant**, sans toucher au fichier d'installation : erreur. Remettez ensuite le script normal par GenererScript.

16 VerifierPDF.vi

But : confirmer qu'après une exécution réussie de MATLAB, le fichier attendu existe et n'est pas vide. Le vieux PDF doit avoir été supprimé avant le lancement.

P1 CheminPDF Path L1 ; P2 error in L4 ; P3 error out R4. Garde Error transmet P2. No Error : N1
File/Directory Info, P1→path,P2→error in. N2 Equal To 0? sur la sortie size ; N3 OR(N1.directory,N2). N4
ErreurSi : Condition=N3, Code7007, Message=PDF absent, vide ou remplace par un dossier., error
in=N1.error out ; sortie→P3. Le type de size est celui fourni par NI, sans conversion réductrice. Un fichier
absent provoque l'erreur native de N1.

Cette fonction ne prouve pas la mise en page ni la validité logique. Une grille insoluble avec PDF == Error
== reste une production correcte. Le contrôle visuel et les exemples attendus viennent au chapitre19.

17 BinairoSolver.vi le programme principal

17.1 Construire la face avant

Enregistrez BinairoSolver.vi dans Execution. Ajoutez ces objets ; textes et clusters exacts :

ID	Nom	Type et défaut	Borne
P1	Image PNG	Path commande, vide	L1
P2	Seuils	SeuilsOCR.ctI,98/90/90	L2
P3	Options	OptionsBinairo.ctI,TRUE/TRUE	L3
P4	error in	erreur commande claire	L4
P5	Executable MATLAB	Path commande, vide	T1
P6	Lanceur fourni	Boolean commande,FALSE par défaut	T2
P7	Image originale	2D Picture indicateur,vide	R1
P8	Matrice reconnue	String indicateur multiligne,vide	R2
P9	Diagnostic	String indicateur multiligne,vide	R3
P10	error out	erreur indicateur	R4
P11	Dossier execution	Path indicateur,vide	non connecté
P12	Taille grille	I32 indicateur,0	non connecté
P13	Bitmap	Boolean2D indicateur,vide	non connecté
P14	Script genere	String indicateur multiligne,vide	non connecté
P15	PDF attendu	Path indicateur,vide	non connecté

B1/B2 non attribuées. Alignez P1..P6 à gauche, l'image à droite, les textes et erreurs en dessous. Ajoutez des étiquettes : Résoudre avec MATLAB et Ouvrir le PDF près des options si leurs champs vous semblent trop courts. Conservez les noms internes resoudre/afficherPDF du typedef.

Zone de dessin Image originale : environ650×620 pixels et fond blanc. Clic droit Visible Items : activez les barres de défilement horizontale/verticale si disponibles. Cela affiche les exemples6×6/8×8 à taille lisible sans redimensionner la matrice de pixels. Le PNG sélectionné est un **fichier**, pas un dossier. Le programme s'exécute une fois avec la flèche Run.

17.2 Dossier et validation de l'entrée

- Hors de toute Case : N1 Current VI's Path→N2 Strip Path ; **stripped path** de N2→P11 et fil Dossier utilisé partout. N2.name est inutilisé. Le VI doit avoir été enregistré, sinon ce chemin est indéfini. Aucun chemin personnel constant n'entre dans le diagramme.
- Garde S0 sélectionnée par P4. Dans Error : P7 picture vide ; P8/P9/P14 chaînes vides ; P12 I32 0 ; P13 tableau Boolean2D vide ; P15 Path vide ; P10=P4.
- Dans S0.No Error : N3 Path To String de P1 ; N4 Match Regular Expression avec motif Normal Display `^[^r\n\x00]+[.][pP][nN][gG]$,` offset0,multilineFALSE,ignorecaseFALSE,error in=P4. N5 String Length(N3) ; N6 Not Equal?(N4.offset past match,N5.length)→ErreurSi code7010 message Choisir un

fichier PNG sans caractere de controle. ; N4.error out→ErreurSi.error in. Cela vérifie le nom ; LireImage vérifie le contenu réel.

4. V1 LireImage : PNG=P1,error in=erreur précédente. V1.Image→tunnel Image ; Bitmap→P13 via tunnel et V2 ComputeRowsCols.Bitmap ; V1.error out→V2.error in. V2.n→tunnel Taille. Garde S1 sur V2.error out.
5. S1.Error : MatriceTexte/Diagnostic/Script vides, PDF path vide,error out=V2.error out. Image/Bitmap/Taille restent raccordés depuis V1/V2, ce qui aide à diagnostiquer une géométrie invalide.

17.3 Déterminer les résultats et supprimer les anciens

Dans S1.No Error : N7 Strip Path(P1)→name ; N8 String Length(name) ; N9 Subtract(N8,4) ; N10 String Subset(name,offset0,length=N9) ; N11 Concatenate Strings(N10,.pdf) ; N12 Build Path(base=Dossier,name=N11)→P15 via les tunnels. Cette expression correspond à CheminPdfBinaire.m. Un PNG nommé a.png donne a.pdf.

N13 Build Path(Dossier,solve.m). V3 SupprimerResultat sur N13 avec error in=V2.error out ; V4 SupprimerResultat sur N12 avec error in=V3.error out. Cela protège contre les résultats d'une exécution précédente. Ne supprimez ni PNG ni source MATLAB. CellValue est supprimé par LireCase avant chaque appel OCR.

17.4 Boucle de lignes puis boucle de colonnes

Dans S1.No Error, posez F1 For Loop extérieur ; V2.n→F1.N. **Aucun auto-indexing**, aucun parallélisme, aucun arrêt conditionnel. Ajoutez trois Shift Registers :

- matrice String, initialisé avec chaîne vide ;
- diagnostic String, initialisé avec chaîne vide ;
- erreur cluster, initialisé avec V4.error out.

Faites entrer par tunnels non indexés Bitmap, Lignes, Dossier, Seuils, n. À l'intérieur F1, posez F2 For Loop ; n→F2.N. Trois Shift Registers : ligneTexte chaîne vide, diagnostic initialisé avec F1.diagnostic gauche, erreur initialisé avec F1.erreur gauche. Faites entrer Bitmap,Lignes,Dossier,Seuils et **F1.i comme indiceLigne** par tunnels non indexés. F2.i est l'indiceColonne.

À l'intérieur F2 : Case C1 sélectionnée par son registre erreur gauche.

C1.Error : ligneTexte gauche→tunnel ligneTexte ; diagnostic gauche→tunnel diagnostic ; erreur gauche→tunnel erreur. Pas de sous-VI d'OCR dans cette case.

C1.No Error :

1. B1 Bundle By Name, cluster de base créé depuis LireCase.Indices, ordre ligne puis colonne ; F1.i transmis→ligne ; F2.i→colonne.
2. V5 LireCase : Bitmap/Lignes/Dossier/Seuils depuis leurs tunnels ; B1→Indices ; F2.erreur gauche→error in.
3. N14 Equal?(F2.i,0) ; N15 Select s=N14,t=chaîne vide,f=chaîne contenant **un espace**.
4. N16 Concatenate Strings trois entrées : F2.ligneTexte gauche, N15, V5.ValeurMatlab. Résultat = proposition de nouvelle ligne.
5. N17 Concatenate Strings trois entrées : F2.diagnostic gauche, V5.Diagnostic, constante LF (\n en Backslash Codes Display).
6. Case C2 sélectionnée par V5.error out. C2.No Error→N16 vers sortie ligneTexte. C2.Error→F2.ligneTexte gauche vers sortie ligneTexte (ne pas ajouter de fausse valeur). Dans **les deux** cases C2 : N17→diagnostic, V5.error out→erreur.

7. C2 sorties→C1 sorties ; C1 ligneTexte/diagnostic/erreur→les trois registres droits de F2. Tous les registres sont alimentés à chaque tour.

Après F2 mais encore dans F1 : N18 Concatenate Strings trois entrées : F1.matrice gauche, **valeur finale de F2.ligneTexte**, constante ;\n saisie en Backslash Codes Display. N18→F1.matrice droite ; F2.diagnostic final→F1.diagnostic droite ; F2.erreur final→F1.erreur droite.

Une erreur peut laisser une matrice partielle affichée pour diagnostic, mais elle empêche GenererScript/LancerMatlab grâce aux gardes. Les tours restants des boucles ne font plus d'accès disque ou de processus. Les boucles n'ont pas besoin d'un arrêt anticipé pour être sûres.

17.5 Générer lancer vérifier

Après F1, toujours dans S1.No Error :

Sous-VI ou nœud	Câblage
V6 GenererScript	MatriceTexte=F1.matrice final ; PNG=P1 ; Options=P3 ; Dossier=N2.stripped path ; error in=F1.erreur final
V7 LancerMatlab	Dossier=N2.stripped path ; ExecutableMatlab=P5 ; LanceurFourni=P6 ; error in=V6.error out
V8 VerifierPDF	CheminPDF=N12.path ; error in=V7.error out
N19 Concatenate Strings	F1.diagnostic final puis V7.Diagnostic

F1.matrice final→Matrice reconnue ; V6.Script→Script genere ; N19→Diagnostic ; V8.error out→P10, en passant par les tunnels S1 puis S0. P7/P13/P12 reçoivent les valeurs Image/Bitmap/n de l'étape17.2 via S0. P15 reçoit N12.path via S1/S0. **Un chemin affiché dans PDF attendu n'est pas un indicateur de succès** : seul error out clair après VerifierPDF autorise à considérer le nouveau fichier produit.

Pour rendre la panne immédiatement visible, ajoutez une LED Erreur non connectée au connector pane : Unbundle By Name status sur le fil final error out→LED. Ajoutez près du cluster : Si Erreur est allumée, ne pas utiliser les anciens fichiers. Ne masquez pas le code ou la source du cluster erreur.

Enregistrez. La flèche doit être entière. Sélectionnez le PNG6×6 et matlab.exe ; choisissez le mode direct (Lanceur fourni=FALSE). Seuils98/90/90, deux options TRUE. Lancez **une fois**. Attendez la fin avant de recliquer. La reconnaissance remplace Cell.bin à chaque cellule : seule la dernière reste disponible à la fin.

18 Lire les résultats attendus

18.1 Exemple 6 par 6

Attendez Taille grille=6 et la matrice suivante dans le script (espaces et fins de lignes peuvent différer, pas les valeurs) :

```
B = [  
NaN 0 NaN NaN NaN NaN;  
NaN NaN 0 NaN 0 NaN;  
NaN NaN NaN NaN 1 0;  
1 1 NaN NaN NaN NaN;  
NaN 0 NaN 0 NaN NaN;  
NaN NaN NaN NaN 0 NaN;  
];  
fichierSource = 'Binairo_6x6.png';  
printGrid(B);  
if true  
    [S, ok] = SolveBinairo(B);  
else  
    S = B;  
    ok = GrilleBinairoValide(B);  
end  
DisplayBinairo(B, S, fichierSource, ok);  
if true  
    uiopen(CheminPdfBinairo(fichierSource), 1);  
end  
if ok  
    fprintf('Binairo traite.\n');  
else  
    fprintf('Grille contradictoire ou sans solution.\n');  
end
```

Le PDF doit contenir tous les indices initiaux en noir gras et toutes les valeurs ajoutées en bleu gras. La date/heure doit correspondre à cette exécution. Fermez le lecteur PDF avant un nouvel essai si Windows verrouille le fichier : une erreur d'accès ne doit jamais être ignorée.

18.2 Quatre combinaisons des options

Résoudre	Ouvrir PDF	Résultat attendu
TRUE	TRUE	PDF de la solution produit et ouvert
TRUE	FALSE	PDF de la solution produit, pas ouvert automatiquement
FALSE	TRUE	PDF de la grille initiale produit et ouvert, cases vides conservées
FALSE	FALSE	PDF de la grille initiale produit, pas ouvert automatiquement

Dans les deux dernières lignes, aucune existence de solution n'est certifiée. Pour une grille initiale déjà contradictoire, le PDF doit afficher == Error ==. L'option Résoudre FALSE ne veut pas dire « ne pas lancer MATLAB » : MATLAB reste responsable du dessin.

18.3 Tester l'ouverture PDF avec le lancement réel

uiopen(chemin,1) est proposé dans l'addendum du cours (P0.PPI_Projet_addendum.25.r2.pdf, diapositive « Open a pdf file »). Son ouverture du lecteur PDF en mode -batch doit néanmoins être testée sur la VM : elle n'a pas été exécutée ici.

1. Fermez tout PDF ouvert. Dans GenererScript.vi, générez le script du checkpoint2x2 avec Résoudre=TRUE, Ouvrir PDF=FALSE et le nom Grille.png.
2. Lancez-le par LancerMatlab.vi en mode direct, avec le vrai matlab.exe. Attendez la fin. Vérifiez un code de retour 0 et l'existence de Grille.pdf. Aucun lecteur ne doit s'ouvrir automatiquement.
3. Régénérez le même script avec Ouvrir PDF=TRUE. Relancez par le même VI. Vérifiez le code 0 **et l'ouverture visible du PDF** ; un fichier créé sans lecteur ouvert ne valide pas l'option.
4. Répétez les deux essais avec Résoudre=FALSE, puis depuis un dossier contenant des espaces. Utilisez aussi le nom L'exemple_test.png pour vérifier les apostrophes et le caractère souligné.
5. Notez la version MATLAB, le code de retour, la sortie d'erreur et l'observation du lecteur. Si le PDF existe mais ne s'ouvre pas, vérifiez d'abord son ouverture par double-clic dans Windows. Ne marquez pas V05 réussi tant que les quatre combinaisons ne fonctionnent pas par le lancement -batch réel.

18.4 Deux types d'échec à ne pas confondre

- Binaire_4x4_Bad.png peut avoir une géométrie correcte tout en étant contradictoire : OCR complet, MATLAB terminé, PDF == Error == attendu.
- Binaire_5x6_Bad.png a un nombre de lignes/colonnes incompatible : erreur de géométrie **avant** l'OCR et avant le lancement MATLAB.

19 Batterie finale de validation native

Pour chaque ligne : notez date, version des logiciels, PASS/FAIL et observation réelle dans Preuves/validation.txt. Ne cochez pas une ligne parce qu'elle est écrite dans la recette. Restaurez les noms/options après chaque essai.

Essai	Action exacte	Réussite attendue
V01	PNG6×6,98/90/90,TRUE/TRUE	matrice du chapitre18, PDF complet, erreur claire
V02	PNG8×8, mêmes options	n=8, tous les indices conservés, règles de la solution vérifiées
V03	PNG4×4_Bad	n=4, PDF == Error == ; aucun faux résultat résolu
V04	PNG5×6_Bad	erreur géométrie, pas de lancement MATLAB
V05	Les quatre lignes du tableau18.2, par le lancement -batch réel ; suivre18.3	code0, PDF correct, ouverture visible seulement si demandée
V06	Sélectionner un chemin PNG inexistant	erreur de lecture visible
V07	Dans un dossier d'essai, créer un texte faux.png contenant abc et le sélectionner	erreur PNG visible, aucun lancement
V08	Champ PNG vide puis fichier .jpg	erreur avant traitement
V09	Renommer votre copie OCR.exe en OCR_absent.exe ; lancer6×6	erreur de lancement OCR, aucun ancien CellValue exploité ; remettre OCR.exe
V10	Seuil vide=100,zero=100,un=100 sur6×6	erreur OCR5 traduite en erreur LabVIEW7006, coordonnées visibles, aucun script partiel lancé
V11	Seuil -1 puis101 puisNaN	erreur de seuil ; remettre98/90/90
V12	Fermer le programme ; créer un dossier nommé CellValue.txt dans un dossier Execution de test	erreur sans suppression du dossier ; l'enlever manuellement après l'essai
V13	Créer un dossier nommé solve.m dans ce même dossier d'essai	erreur avant MATLAB, dossier conservé
V14	Créer un dossier nommé Binaire_6x6.pdf dans le dossier d'essai	erreur avant lancement, dossier conservé
V15	Mode direct : sélectionner un matlab.exe inexistant	erreur lancement visible et pas de succès PDF
V16	Essai isolé de LancerMatlab avec solve.m contenant error('Essai volontaire');	erreur script réelle détectée ; régénérer ensuite le script normal
V17	Essai isolé : supprimer le PDF, faire lancer un script qui ne produit aucun PDF, puis VerifierPDF	erreur7007 ou erreur native fichier absent
V18	Relancer V01 juste après un échec	succès nouveau, pas de reprise de fichier périmé
V19	Fermer LabVIEW, copier Execution dans un autre dossier avec espaces, rouvrir BinaireSolver.vi et sélectionner PNG	succès identique, pas de chemin absolu figé
V20	Dans ce dossier, rendre un fichier résultat non inscriptible avec les permissions du compte de test, puis lancer	erreur écriture transmise ; restaurer les permissions

Essai	Action exacte	Réussite attendue
V21	Tous les checkpoints EtendueNoire/IntervallesNoirs/rectangle et binaire asymétrique	indices et octets exacts
V22	Mode direct : succès et erreur volontaire, avec attente effective ; répéter pour le mode fourni seulement s'il est installé	attend la fin et transmet l'erreur ; captures conservées

Pour V20, l'attribut « lecture seule » d'un dossier Windows ne prouve pas une interdiction d'écrire : utilisez un fichier protégé ou les permissions réelles, sans modifier les permissions de l'installation de cours. Si votre compte ne permet pas ce test, marquez-le non exécuté et utilisez V12-V14 pour vérifier déjà les échecs d'ouverture.

Pour V02, comparez l'original à la matrice reconnue avant de juger la solution. Pour chaque ligne et colonne de la solution : moitié0/moitié1, jamais trois identiques consécutifs, aucune ligne complète dupliquée, aucune colonne complète dupliquée. La photo de l'image d'origine et le PDF final doivent permettre de vérifier les couleurs des indices.

19.1 Ajouter des tests de géométrie sans écrire de code externe

Dans un VI de test séparé : Initialize Array de Boolean FALSE de taille 46×46, puis Replace Array Subset pour dessiner des traits complets à lignes/colonnes 0,15,30,45 avec TRUE (utilisez des vecteurs TRUE produits par Initialize Array de longueur 46). Le tableau représente 3×3 cases et doit être refusé pour nombre impair. Répétez avec dimensions 31×46 et traits de lignes 0,15,30, colonnes 0,15,30,45 : 2×3 doit être refusé. Une matrice totalement FALSE doit être refusée comme absence de grille. Les instructions géométrie donnent aussi des tests unitaires plus simples de transitions.

Si vous créez des images complémentaires avec un logiciel de dessin, gardez-les comme tests, sans les confondre avec les fichiers du professeur. Une grille photographiée en perspective n'est pas un cas implicitement pris en charge : l'algorithme demandé suppose des traits alignés avec les axes.

20 Si quelque chose ne marche pas

Ne modifiez pas dix fils à la fois. Arrêtez-vous au **premier checkpoint faux**.

Symptôme	Vérification dans cet ordre
Flèche cassée	Error List ; borne non câblée ; type exact ; sortie Case oubliée ; tunnel indexé par erreur
Bitmap inversé	chapitre 7.4 et les deux pixels de calibration ; ne pas inverser dans plusieurs VIs
n vaut 0,1 ou nombre incohérent	Bitmap correct ? marges prises en compte ? balayage top+10/left+10 ? Intervalles Noirs inclut-il les bords ?
Cell.bin refusé	lire les huit premiers U8 ; largeur/hauteur dans cet ordre ; byte order 2 ; deux prepend FALSE ; pixels U8 et non bits compactés
OCR code 5 partout	polarité ; dimensions ; découpage hors traits ; seuils 98/90/90 ; police fournie présente
Fichier OCR jamais trouvé	working directory de System Exec ; OCR.exe réellement Windows ; nom exact Cell.bin ; file close avant appel
Matrice transposée	F1.i=ligne, F2.i=colonne ; Array Subset row puis col ; ne pas transposer le bitmap de production
MATLAB rejette les options	le script doit contenir true et false, pas 0/1 ni "TRUE"
MATLAB fonction introuvable	treize fichiers .m dans Execution ; working directory du lanceur ; nom/casse du fichier
PDF ancien malgré panne	supprimer l'ancien avant lancement ; attendre la fin ; vérifier error out, return code et fichier nouveau
PDF verrouillé	fermer le lecteur de PDF avant de réessayer ; ne pas ignorer l'erreur de suppression
Le lanceur fourni a d'autres bornes	capture Front Panel, connector pane, Context Help et diagramme ; ne pas adopter les bornes du Billard

20.1 Ce qu'il faut renvoyer pour une correction précise

1. Enregistrez tous les VIs. Ne relancez pas une autre image avant d'avoir copié les résultats : cela écraserait Cell.bin/CellValue.txt.
2. Copiez dans un dossier Preuves daté : VIs et .ctl actuels, PNG exact utilisé si partage permis, Cell.bin, CellValue.txt si présent, solve.m si présent et PDF si présent.
3. Ajoutez une capture de la face avant principale et du diagramme du premier sous-VI défaillant. Les identifiants d'objets doivent rester lisibles.
4. Copiez intégralement error out.status/code/source et Diagnostic dans un texte. Une seule capture « erreur » sans code ni source ne suffit pas.
5. Pour un fil impossible, ajoutez Context Help du nœud et de la borne visée. Pour une panne de lancement, ajoutez le texte exact de la commande, return code, stdout, stderr.
6. Fermez LabVIEW puis archivez ce dossier de preuves. Le dossier ne constitue pas encore l'archive de remise.

21 Terminer et préparer la remise ultérieure

- Le programme principal s'appelle **BinairoSolver.vi**, s'ouvre sans dépendance manquante et fonctionne avec la flèche Run.
- `ComputeRowsCols.vi` et `ComputeCellRect.vi` existent avec ces noms exacts ; les autres sous-VIs sont ceux de cette recette.
- Tous les VIs ont une description utile et les trois auteurs. Les boucles/cases délicates sont étiquetées ; les fils restent lisibles.
- Remettez les contrôles de test à leurs valeurs normales ; fixez les seuils98/90/90 provisoires et les optionsTRUE/TRUE. Sauvegardez les valeurs par défaut. Les chemins de votre installation ne doivent pas être inscrits comme constantes dans le diagramme.
- Validez le lancement direct MATLAB, la grille6×6, la grille8×8, la grille impossible, les erreurs et la copie du dossier avant d'affirmer que LabVIEW est terminé.
- Vérifiez la **dernière** annonce Moodle : connecteur principal Binairo éventuel, liste finale d'erreurs, forme et date de remise. Si un connecteur officiel est publié, seuls les raccordements du panneau principal à ce connecteur doivent être adaptés ; ne remplacez pas les algorithmes par le Billard.
- Préparez plus tard l'archive avec tous les VIs, typedefs, fonctions MATLAB, sources C, police fournie et exécutable Windows réellement testé. Le dossier source-only actuel n'est pas, à lui seul, une archive finale de remise.
- Le rapport court, les résultats natifs et le transcript demandé sont des pièces séparées à préparer ensuite. Cette recette de construction ne les remplace pas.
- Les modifications faites uniquement dans la VM ne sont pas sauvegardées par magie dans GitHub. Fermez LabVIEW et ramenez les `.vi/.ctl/.lvproj` modifiés, ou poussez-les vers le dépôt privé autorisé. Ne remplacez jamais un VI binaire modifié par un autre membre sans conserver les deux versions.

22 Références à consulter en cas de différence

Les numéros suivants sont les **pages physiques du PDF**, pas nécessairement les nombres imprimés sur les diapositives.

Source du cours	Pages	Exigence utilisée
P00.PPI_Projet.2026.2.pdf	6,15-18,23-27,31-36	noms, format binaire/texte, interface, orchestration, résolution, PDF, erreurs
Exo.Proj.Binaire.LabVIEW.1.pdf	3-10,13-16	lecture/conversion image, indexation, co-localisation, System Exec, géométrie
Exo.Proj.Binaire.LabVIEW.2.pdf	2-7	matrice générée, script, lanceur fourni, erreurs
Exo.Proj.Binaire.Matlab.1.pdf	1-2	résultat PDF et fonctions d'affichage
P0.PPI_Projet.25.r6.pdf	39-43	critères généraux hérités : erreurs, fonctions, documentation, portabilité
P0.PPI_Projet_addendum.25.r2.pdf	6-7,14,18	vérification automatisée, valeurs par défaut, validation PNG, décimales

Les exercices de ce tableau sont des aides secondaires ; leurs noms de sous-VIs et leur lanceur ne deviennent pas des obligations absentes de la donnée.

Documentation primaire des fonctions NI utilisées, pour vérifier leurs bornes avec Context Help dans votre version :

- [Write to Binary File](#) : refnum partagé, octets et taille préfixée.
- [Unflatten Pixmap](#) : sortie selon profondeur et indices de palette.
- [Picture to Pixmap](#) : profondeur, rectangle et fond.
- [Lancer directement un exécutable](#) : chemin complet de l'exécutable avec System Exec.
- [System Exec et commandes](#) : lancement et attente.
- [Codes de localisation](#) : forcer le point avec % ; .
- [Lancement MATLAB sous Windows](#) : options -wait et -batch.
- [Match Regular Expression](#) : sous-chaînes capturées et fin du match.
- [Color to RGB](#) : composantes U8 à convertir avant addition.
- [File/Directory Info](#) et [Delete](#) : contrôle des fichiers et suppression non récursive.

23 Contrôle de couverture avant de déclarer la partie terminée

La donnée 2026.2 prime sur chaque exercice. Les pages sources sont répertoriées au chapitre 22. Les critères généraux de 2025 sont repris parce que leur application à 2026 a été confirmée ; les exigences techniques du Billard ne le sont pas. Ce tableau relie chaque exigence disponible à sa construction et à sa vérification.

Exigence	Où la construire	Preuve à obtenir
VI principal BinaïroSolver.vi et interface utilisable	17.1	source PNG, deux options et erreur visibles ; V01/V05
Lecture PNG et conversion native couleur→noir/blanc	7	dimensions, polarité et image affichée ; checkpoint 7.5/V07
Nombre pair et égal de lignes/colonnes ; traits aux bords	annexe G	V04/V21, grille synthétique 2×2, refus 3×3 et 2×4
Comptage et découpage de la donnée, noms de sous-VIs repris de l'exercice	annexe G	résultats nommés et coordonnées mesurées ; V21
Cellules sans bordures ; format binaire correct	9/12	asymétrie 35×40, octets attendus, cellule 72×68
Appel C paramétré, séquentiel, résultat réellement lu	10-12/17.4	V01, V09, V10 ; matrice comparée à l'image
Script MATLAB produit depuis toute la grille reconnue	13/17	solve.m réellement généré, valeurs 0/1/NaN exactes
Lancement MATLAB et détection des erreurs (donnée p26,36)	15.2 ; option 15.3 seulement si retenue	attente effective, succès et panne volontaire ; V22
Résolution et affichage commandés par les deux options	13/18.2	les quatre combinaisons V05
PDF au nom du PNG, style et résultat conformes	16/18	contrôle visuel des indices/couleurs/titre/date, V01-V03
Pas de PNG, OCR absent/en erreur, écriture script impossible, MATLAB absent/script en erreur	5/6/12/15/17	V06-V17 : erreur affichée, aucune étape suivante exécutée
Pas de reprise de résultats périmés	6/12/16/17	V09, V12-V18, nouveau PDF après succès
Chemins portables et fichiers co-localisés	1/17.2	V19 dans un autre dossier avec espaces
Sous-VIs, connecteurs, types et valeurs par défaut cohérents	2/3, chaque interface	absence de flèche cassée ; ouvrir chaque sous-VI et contrôler ses défauts
Documentation et auteurs ; diagrammes lisibles	2.6/21	aide de chaque VI, contrôles décrits, fils rangés
Exécution effective dans votre environnement	19	résultats natifs enregistrés pour tous les tests applicables
Derniers compléments de consignes/erreurs/connecteur	21	comparaison avec Moodle avant remise, adaptation si un contrat nouveau est fourni

État de cette recette : le C actuel a été compilé localement et les 116 cellules des trois PNG carrés ont été découpées par un contrôle indépendant puis soumises à cet exécutable avec 98/90/90. Les matrices obtenues concordent avec les exemples. Cela vérifie le contrat découpage/OCR, **pas les VIs natifs**, ni le binaire Windows, ni l'exécution MATLAB. Ces validations restent celles du chapitre 19, à effectuer par vous.

La partie LabVIEW peut être déclarée terminée lorsque les tests natifs sont réussis, le mode direct de lancement MATLAB fonctionne réellement, les fichiers nécessaires sont sauvegardés, et les dernières consignes officielles sont intégrées. L'inspection facultative du chapitre 14 n'est pas une condition de conformité à la donnée.

23.1 Question à transmettre au professeur si elle n'a pas encore reçu de réponse

Bonjour,

Pour notre projet Binairo 2026, pourriez-vous confirmer trois points ?

1. Nous avons appliqué la procédure détaillée de la donnée 2026.2, page 18 : score $\geq$ seuil, puis maximum de score - seuil. Sa page 10 évoque le score brut. Pouvez-vous confirmer que la procédure détaillée fait foi et préciser le départage lorsque deux marges sont exactement égales ?
2. Existe-t-il un connecteur imposé pour BinairoSolver.vi cette année ? Si oui, pourriez-vous nous transmettre le fichier ou le schéma avec les types attendus ?
3. La liste définitive des erreurs à traiter et les dernières consignes de remise sont-elles disponibles ?

Merci beaucoup.

Continuez les étapes de construction indépendantes de ces réponses. Toute règle nouvelle devra ensuite être répercutée dans le composant concerné et ses essais avant remise.

Annexe G — Détecter la grille et découper ses cases

Ce chapitre construit `EtendueNoire.vi`, `IntervallesNoirs.vi`, `ComputeCellRect.vi`, puis `ComputeRowsCols.vi`, dans cet ordre. Les deux derniers noms sont repris de l'exercice comme conventions compatibles. La donnée impose les opérations correspondantes (p25-27), sans fixer les noms de ces deux sous-VIs. Les deux premiers sont nos sous-VIs auxiliaires. `ErreurSi.vi` doit déjà exister : entrées `Condition Boolean`, `Code I32`, `Message String`, `error in` ; sortie `error out`. Il conserve une erreur entrante ; sinon il crée l'erreur demandée uniquement lorsque `Condition` est vraie.

Les opérations ci-dessous se font dans LabVIEW. Elles ne sont pas des tests déjà exécutés. Les résultats indiqués sont les résultats à obtenir avant de poursuivre.

Références : `Exo.Proj.Binaire.LabVIEW.1.pdf`, pages physiques 5-8 et 15-16 ; `P00.PPI_Projet.2026.2.pdf`, pages physiques 25-27. Le professeur demande de compter les traits, traiter ceux qui touchent le bord, vérifier un nombre pair et égal de lignes/colonnes, puis tenir compte des bordures lors du découpage. Nous mesurons chaque trait : une bordure de 5 pixels est donc prise en compte par sa largeur réelle, sans soustraire arbitrairement 5 partout.

G0. Conventions à appliquer à chaque fil

1. Enregistrer les nouveaux fichiers dans le même dossier de travail LabVIEW que le VI principal.
2. Bitmap est le tableau **Boolean 2D déjà normalisé** fourni par LireImage.vi : TRUE = noir, FALSE = blanc. Premier indice = ligne, deuxième indice = colonne. Toutes les coordonnées commencent à **0**.
3. Tous les nombres de ce chapitre sont des **I32**, y compris les constantes. Pour chaque constante numérique : clic droit → Representation → I32.
4. Les intervalles sont [debut, fin) : debut appartient au trait ; fin est le premier pixel après le trait. De même, droite et bas ne font pas partie du rectangle extrait. Largeur = droite–gauche ; hauteur = bas–haut. **Ne pas ajouter 1.**
5. Pour placer une primitive, Ctrl+Space, saisir son nom anglais imprimé ici, sélectionner le résultat exact, Entrée. Ctrl+H ouvre l'aide contextuelle. Les noms de ports priment sur leur position dans l'icône.
6. Dans les tableaux, A → B, C signifie deux branches issues du même fil. .x/.y sont les deux entrées d'une opération ; .sortie son unique résultat. Index Array.tableau/index/element désigne les ports natifs correspondants. Pour une matrice, index0 est la ligne et index1 la colonne.
7. Un calcul nommé dans un tableau est **un objet à placer**. Un test nommé est **une primitive de comparaison distincte**. Les constantes peuvent être branchées vers plusieurs objets. Les noms de repères servent d'étiquettes libres dans le diagramme, pas de noms à rechercher dans Quick Drop.
8. Pour chaque Case Structure, connecter le **cluster d'erreur complet** au sélecteur. Les cas s'appellent alors Error et No Error. **Ne pas sélectionner Use Default If Unwired**. Tous les tunnels de sortie doivent être câblés dans les deux cas.
9. Pour acheminer une valeur extérieure jusqu'à un objet situé dans un cas imbriqué, tirer directement le fil à travers les bords des structures : LabVIEW crée les tunnels d'entrée nécessaires. **Ne pas utiliser de variables locales**. Une valeur qui ressort d'un cas doit, elle, être câblée dans **tous** les cas.
10. Chaque boucle For Loop a le parallélisme désactivé, aucun terminal d'arrêt et aucun registre à décalage sauf lorsque ce chapitre en demande un. Activer l'indexation uniquement aux tunnels explicitement indiqués.
11. Build Array recevant plusieurs valeurs scalaires a Concatenate Inputs **désactivé**. Lorsqu'on concatène des tableaux, le texte le demande explicitement.
12. Aucun tableau d'indices géométriques n'est lu avant la validation de sa taille et de l'indice.
13. Les codes 6101–6104 sont des **codes de notre projet**, pas des numéros officiels du professeur ou de NI.
14. Pour chaque VI, compléter File → VI Properties → Documentation : rôle, sens des entrées/sorties, unités et erreurs. Ajouter les auteurs Louis Pédelaborde, Romeo Mugnier de Almeida, Raphaël Pical. Sauvegarder des entrées vides/0 et un cluster sans erreur avec Edit → Make Current Values Default, puis sauvegarder le VI.

G1. Vérifier les deux types et attribuer les connecteurs

Les deux fichiers .ctl ont déjà été créés au chapitre3 : vérifiez-les avec les indications ci-dessous, **ne créez pas une seconde copie**. Leur dossier est Execution.

G1.1 LignesGrille.ctl

- 1. Dans une face avant vide, déposer un Cluster.
- 2. À l'intérieur, déposer quatre Array contenant chacun un contrôle numérique I32. Un seul indice affiché : tableaux **1D**.
- 3. Nommer exactement les quatre tableaux debutX, finX, debutY, finY.
- 4. Clic droit sur le bord du cluster → Reorder Controls in Cluster. Affecter respectivement les ordres 0,1,2,3.
- 5. Laisser les quatre tableaux vides : ne pas saisir de zéro dans une case.
- 6. Clic droit → Advanced → Customize. Dans l'éditeur de contrôle, choisir Type Def. ; enregistrer LignesGrille.ctl ; fermer l'éditeur en acceptant de remplacer le contrôle initial.

G1.2 RectangleCellule.ctl

Même procédure, mais cluster de quatre **scalaires I32** : gauche, haut, droite, bas, dans cet ordre. Valeurs par défaut 0. Enregistrer RectangleCellule.ctl. Il s'agit du rectangle **de notre programme**, pas du cluster rectangle natif des fonctions Picture ; ici l'extraction finale se fait avec Array Subset.

G1.3 Connecteurs des quatre sous-VIs

Pour chaque VI, afficher le connecteur en haut à droite de la face avant et choisir le motif de **12 bornes** : quatre à gauche, quatre à droite, deux en haut et deux en bas. Repérer L1-L4 de haut en bas à gauche, R1-R4 de haut en bas à droite, T1/T2 en haut, B1/B2 en bas. Cliquer la borne puis le contrôle/indicateur indiqué.

VI	L1	L2	L3	L4	R1	R2	R3	R4
EtendueNoire.vi	Bits	libre	libre	error in	Debut	Fin	libre	error out
IntervallNoirs.vi	Bits	libre	libre	error in	Debuts	Fins	libre	error out
ComputeCellRect.vi	ligne	colonne	Lignes	error in	Rectangle	libre	libre	error out
ComputeRowsCols.vi	Bitmap	libre	libre	error in	n	Lignes	libre	error out

Toutes les bornes T1,T2,B1,B2 restent libres. Les bornes de données d'entrée utilisées sont Required ; error in Recommended. Ce motif est notre choix pour les **sous-VIs** ; il ne prétend pas reproduire un connecteur officiel du VI principal.

G2. EtendueNoire.vi — trouver le premier et le dernier pixel occupé

G2.1 Face avant

Créer et sauvegarder EtendueNoire.vi. Ajouter :

Nom	Sens	Type	Défaut
Bits	contrôle	Boolean 1D	vide
error in	contrôle	cluster d'erreur standard	aucune erreur
Debut	indicateur	I32	0
Fin	indicateur	I32	0
error out	indicateur	cluster d'erreur standard	aucune erreur

G2.2 Placer et câbler

- Placer S0, une Case Structure commandée par error in. Créer trois sorties : debut, fin, erreur ; relier aux indicateurs correspondants.
- Cas **Error** : constante I32 0 → debut et fin ; error in → erreur.
- Dans **No Error**, placer les objets suivants :

Repère	Primitive	Câblage des entrées	Résultat
A	Array Size	Bits → array	longueur
P	Search 1D Array	Bits → 1D array ; TRUE → element ; 0 → start index	premier indice noir
R	Reverse 1D Array	Bits → array	tableau inversé
Q	Search 1D Array	R.sortie → array ; TRUE → element ; 0 → start index	dernier noir vu depuis la fin
F	Subtract	A.longueur → x ; Q.index → y	fin exclusive
T	Less?	P.index → x ; 0 → y	absence de noir
E	ErreurSi.vi	T.sortie → Condition ; 6102 → Code ; Aucun pixel noir sur l'axe examine. → Message ; error in → error in	erreur validée
S1	Case Structure	E.error out → sélecteur	validation finale

- Dans S1.No Error : P.index → sortie debut ; F.sortie → sortie fin ; E.error out → sortie erreur.
- Dans S1.Error : 0 → debut et fin ; E.error out → erreur.
- Relier les trois sorties de S1 aux trois sorties de S0. Aucun résultat calculé en cas d'erreur ne contourne S1.
- Il n'y a **aucune boucle** ni registre à décalage dans ce VI. Les sorties inutilisées de Search 1D Array restent non connectées.

G2.3 Vérifier avant de continuer

Bits	Debut	Fin	Erreur
F,F,T,T,F,T,F	2	6	aucune
T	0	1	aucune
F,F	0	0	6102
tableau vide	0	0	6102

Injecter également `error in = status TRUE`, code 123, source "essai" : retrouver exactement cette erreur, avec `Debut=Fin=0`. Ne pas remplacer l'erreur 123 par 6102. Sauvegarder.

G3. IntervallesNoirs.vi — relever tous les traits, même aux bords

G3.1 Face avant et structure

Cr  er le VI. Contr  les : Bits Boolean1D vide et error in. Indicateurs : Debuts et Fins, deux tableaux I32 1D vides, et error out. Attribuer le connecteur G1.

Placer S0, Case sur error in, sorties debuts, fins, erreur. Dans Error : tableau I32 1D vide vers debuts et fins, erreur entrante vers erreur.

G3.2 Cas No Error : objets et tous les fils

1. Cr  er une constante **tableau Boolean1D contenant exactement un FALSE**.
2. Placer B = Build Array, agrandir    trois entr  es et activer **Concatenate Inputs**. Câbler : entr  e0 = [FALSE], entr  e1 = Bits, entr  e2 = [FALSE]. Nommer le r  sultat avecBords.
3. Placer A = Array Size sur Bits ; N = Add avec A.sortie en x et constante1 en y.
4. Placer F = For Loop. N.sortie    terminal N. avecBords entre par un tunnel **sans indexation**. Aucun registre.
5. Dans F placer : Add, deux Index Array, deux NOT, deux AND. Câbler exactement :

Objet	Entr��es	R��sultat utilis��
J = Add	i �� x ; 1 �� y	i+1
P = Index Array	avecBords �� array ; i �� index	pr��c��dent
C = Index Array	avecBords �� array ; J.sortie �� index	courant
NP = NOT	P.element	non pr��c��dent
NC = NOT	C.element	non courant
D = AND	NP.sortie �� x ; C.element �� y	d��but de trait
E = AND	P.element �� x ; NC.sortie �� y	fin de trait
6. Tirer le fil i vers **deux tunnels de sortie distincts** de F. Clic droit sur chaque tunnel    Tunnel Mode    **Conditional**. Ces tunnels doivent construire chacun un tableau I32 1D.
7. D.sortie    petit terminal conditionnel du premier tunnel ; E.sortie    condition du deuxi  me tunnel.
8. Premier tableau de sortie F    S0.debuts ; deuxi  me    S0.fins. error in    S0.erreur. Relier les sorties S0 aux indicateurs.

La boucle effectue longueur(Bits)+1 tours, i=0 jusqu'   longueur(Bits) incluse. Le tableau compl  t      longueur(Bits)+2   l  ments ; les lectures i et i+1 sont donc toutes valides. L'indice i est d  j   celui de l'image d'origine : **ne pas retrancher 1**.

G3.3 Tests à réaliser

Bits	Debuts	Fins
F,T,T,F,T,F	1,4	3,5
T,T,F,T	0,3	2,4
T,T	0	2
F,F	vide	vide
vide	vide	vide

Tous ces essais ont un error out sans erreur. Le cas sans trait sera rejeté par ComputeRowsCols, pas ici.
Avec erreur entrante 123 : sorties vides, même erreur123. Sauvegarder.

G4. ComputeCellRect.vi — rectangle intérieur d'une case

G4.1 Face avant

Contrôles ligne l32=0, colonne l32=0, Lignes instance de LignesGrille.ctl vide, error in. Indicateurs Rectangle instance RectangleCellule.ctl à zéro, error out. Attribuer le connecteur G1.

Le rôle exact du VI est de produire :

- gauche = finX[colonne] ; droite = debutX[colonne+1] ;
- haut = finY[ligne] ; bas = debutY[ligne+1].

Il enlève les traits eux-mêmes. La largeur et la hauteur intérieures doivent rester entre **10 et 100 inclus**, conformément au C actuel et aux bornes de contrôle illustrées dans le projet (P00 p19). Le maximum 4096 pour une coordonnée est une garde de notre projet cohérente avec la lecture d'image, pas une constante prescrite par le professeur.

G4.2 Premier filtre : ne jamais indexer hors des tableaux

1. Placer S0 Case sur error in, sorties rectangle, erreur.
2. S0.Error : rectangle constant(0,0,0,0) → rectangle ; error in → erreur.
3. S0.No Error : placer Unbundle By Name sur Lignes, agrandir à quatre champs exacts debutX,finX,debutY,finY.
4. Placer quatre Array Size, un par champ. Nommer les longueurs AX,BX,AY,BY.
5. Placer Subtract(AX,1), résultat n ; Quotient & Remainder(n,2), utiliser seulement remainder.
6. Placer les **neuf** comparateurs suivants, chaque ligne étant une primitive et ses deux entrées :

Test	Primitive	x	y
T1	Not Equal?	AX	BX
T2	Not Equal?	AX	AY
T3	Not Equal?	AY	BY
T4	Less?	AX	3
T5	Less?	ligne	0
T6	Less?	colonne	0
T7	Greater Or Equal?	ligne	n
T8	Greater Or Equal?	colonne	n
T9	Not Equal?	remainder	0

7. Build Array neuf entrées scalaires, T1..T9 dans cet ordre → Or Array Elements → ErreurSi.Condition. Code6104, Message Indices de case ou tableaux de traits invalides., error in = erreur entrante.
8. Placer S1 Case sur ce error out, avec sorties rectangle/erreur. S1.Error : rectangle zéro et erreur venant d'ErreurSi. Relier sorties S1 vers sorties S0.

G4.3 Dans S1.No Error uniquement : calcul puis second filtre

1. Placer Add(colonne,1) et Add(ligne,1).

2. Placer quatre Index Array :

Champ tableau	Indice	Nom du résultat
finX	colonne	gauche
debutX	colonne+1	droite
finY	ligne	haut
debutY	ligne+1	bas

3. Subtract(droite,gauche) → largeur ; Subtract(bas,haut) → hauteur.

4. Placer huit tests, avec constantes I32 : largeur<10 ; largeur>100 ; hauteur<10 ; hauteur>100 ; gauche<0 ; haut<0 ; droite>4096 ; bas>4096. Utiliser Less? ou Greater? selon le signe indiqué.

5. Ces huit sorties Boolean → Build Array huit entrées → Or Array Elements → deuxième ErreurSi.Condition. Code6104, Message Rectangle de case invalide ou dimensions hors de 10 a 100 pixels. ; premier ErreurSi.error out → deuxième ErreurSi.error in.

6. Placer Bundle By Name, brancher un rectangle constant zéro à son entrée cluster ; choisir quatre champs gauche,haut,droite,bas ; brancher chaque résultat du point2 au champ de même nom.

7. Placer S2 Case sur deuxième ErreurSi.error out, sorties rectangle/erreur. Dans No Error : cluster calculé vers rectangle, deuxième erreur vers erreur. Dans Error : rectangle zéro vers rectangle, deuxième erreur vers erreur.

8. Sorties S2 → sorties S1 → sorties S0 → indicateurs. Aucune boucle et aucun registre.

G4.4 Essais précis

Saisir debutX=[0,20,40], finX=[2,22,42], debutY=[0,18,36], finY=[2,20,38].

ligne,colonne	Rectangle (gauche,haut,droite,bas)	Résultat
0,0	2,2,20,18	largeur18 hauteur16, aucune erreur
1,1	22,20,40,36	largeur18 hauteur16, aucune erreur
2,0	0,0,0,0	erreur6104
0,-1	0,0,0,0	erreur6104

Raccourcir seulement finX d'un élément : erreur6104. Remplacer debutX[1] par11 : largeur9 pour case0,0, erreur6104. Essayer largeur10 puis100 : acceptées ; largeur101 : rejetée. Erreur entrante123 : rectangle nul et même erreur123. Sauvegarder.

G5. ComputeRowsCols.vi — nombre de cases et traits mesurés

G5.1 Face avant et squelette des cas

Contrôles Bitmap Boolean2D vide, error in. Indicateurs n I32=0, Lignes LignesGrille.ctl vide, error out. Attribuer connecteur G1.

Créer S0 Case sur error in. Dans son No Error seront imbriqués successivement S1,S2,S3,S4. **Chaque Case a exactement trois sorties** : n, lignes, erreur. Dans le cas Error de chacune : I32 0 → n ; cluster LignesGrille dont les quatre tableaux sont vides → lignes ; erreur qui sélectionne ce Case → erreur. Dans le No Error, les trois sorties du Case intérieur rejoignent les trois sorties du Case extérieur. Seul S4.No Error fournit les résultats finaux valides. S0.sorties vont aux indicateurs.

Cette règle définit explicitement les cinq branches d'erreur. Il faut les câbler toutes ; aucun tunnel de sortie par défaut.

G5.2 S0.No Error : dimensions avant toute opération

1. Array Size(Bitmap) → tableau dimensions ; deux Index Array aux indices0 et1 donnent hauteur et largeur.
2. Quatre tests : hauteur<32, hauteur>4096, largeur<32, largeur>4096.
3. Build Array des quatre tests → Or Array Elements → ErreurSi.Condition ; Code6101 ; Message Dimensions d'image hors de 32 a 4096 pixels. ; error in entrante → error in.
4. ErreurSi.error out → sélecteur S1 et son fil d'erreur. Bitmap entre dans S1 sans indexation. Le domaine32..4096 est notre garde pour les images, distinct des dimensions10..100 des cellules. Ne pas imposer hauteur=largeur ici.

G5.3 S1.No Error : trouver la position réelle du bord noir

Placer ces objets :

Repère	Objet	Configuration et fils
F1	For Loop	Bitmap entre avec auto-indexation : un tableau1D ligne par tour ; N non câblé ; aucun registre
O1	Or Array Elements, dans F1	ligne → entrée ; sortie scalaire vers tunnel de sortie F1 auto-indexé
T	Transpose 2D Array	Bitmap → entrée ; sortie vers F2
F2	For Loop	transposé entre auto-indexé : une colonne d'origine par tour ; N non câblé ; aucun registre
O2	Or Array Elements, dans F2	colonne → entrée ; sortie scalaire vers tunnel de sortie F2 auto-indexé
V1	EtendueNoire.vi	F1.tableau sortie → Bits ; erreur de S1 → error in
V2	EtendueNoire.vi	F2.tableau sortie → Bits ; V1.error out → error in
A1	Add	V1.Debut → x ; 10 → y ; résultat ligneAnalyse
A2	Add	V2.Debut → x ; 10 → y ; résultat colonneAnalyse
C1	Greater Or Equal?	ligneAnalyse → x ; V1.Fin → y
C2	Greater Or Equal?	colonneAnalyse → x ; V2.Fin → y

Repère	Objet	Configuration et fils
O3	OR	C1.sortie → x ; C2.sortie → y
E2	ErreurSi.vi	O3 → Condition ; 6102 → Code ; Les lignes d'analyse a bord+10 sont hors de la grille. → Message ; V2.error out → error in

E2.error out → sélecteur S2 et son fil d'erreur. Passer Bitmap, ligneAnalyse et colonneAnalyse à l'intérieur de S2 par des tunnels d'entrée.

Ne pas remplacer Bitmap par le tableau transposé : la transposition ne sert qu'à calculer l'occupation des colonnes. La grille peut avoir une marge blanche ; bord+10 n'est donc pas nécessairement l'indice absolu10. Une image entièrement blanche donne6102 et n'atteint jamais les Index Array suivants.

G5.4 S2.No Error : extraire les axes et compter les traits

1. Deux Index Array reçoivent **Bitmap original**. Agrandir pour voir les indices ligne/colonne.
2. Premier : ligneAnalyse → index0 ; **index1 volontairement non câblé**. Sortie tableau1D = parcours horizontal.
3. Deuxième : **index0 volontairement non câblé** ; colonneAnalyse → index1. Sortie tableau1D = parcours vertical.
4. Placer V3,V4 = deux IntervallesNoirs.vi. Horizontal → V3.Bits ; vertical → V4.Bits ; erreur S2 → V3.error in → V4.error in via V3.error out.
5. Bundle By Name sur cluster LignesGrille vide : V3.Debuts → debutX ; V3.Fins → finX ; V4.Debuts → debutY ; V4.Fins → finY. Résultat nommé lignesCalculees.
6. Pour X : Array Size(V3.Debuts) et Array Size(V3.Fins)→Add ; cette somme est le nombre de transitions. Quotient & Remainder(somme,2).quotient→Subtract(quotient,1)→nombreColonnes. Pour Y : mêmes cinq nœuds avec V4.Debuts et V4.Fins→nombreLignes. Constantes2 et1 en I32. Cela applique littéralement transitions/2−1 (donnée p27). Les restes de division sont nuls puisque IntervallesNoirs apparie les débuts et fins.
7. Quotient & Remainder(nombreColonnes,2), utiliser remainder.
8. Trois tests : nombreColonnes != nombreLignes ; nombreColonnes<2 ; remainder !=0. Les relier à Build Array trois entrées → Or Array Elements → ErreurSi.Condition.
9. ErreurSi.Code=6103 ; Message=La grille doit avoir au moins deux lignes, autant de colonnes et une taille paire. ; V4.error out → error in.
10. ErreurSi.error out → sélecteur S3 et son fil d'erreur ; nombreColonnes → entrée n de S3 ; lignesCalculees → entrée lignes de S3.

Les débuts et fins ont déjà été appariés par IntervallesNoirs : leur nombre est le **nombre de traits**. Un début et une fin forment deux transitions : additionner leurs nombres, diviser par2 puis retrancher1 donne le nombre de cases. Ne divisez pas seulement le nombre de débuts par2.

G5.5 S3.No Error : vérifier toutes les largeurs et hauteurs

1. Placer une For Loop F3 ; n → N. Aucun arrêt conditionnel ni parallélisme.
2. Ajouter **un registre à décalage de cluster d'erreur** : clic droit sur bord gauche → Add Shift Register. Depuis l'extérieur, erreur S3 → registre gauche d'initialisation. Ce fil est obligatoire.
3. Lignes entre dans F3 par tunnel **sans indexation**. Constante0 également sans indexation si placée à l'extérieur.
4. À l'intérieur, placer R1 et R2, deux ComputeCellRect.vi.

5. R1 : ligne=0, colonne=i, Lignes=Lignes, error in=registre gauche courant.
 6. R2 : ligne=i, colonne=0, Lignes=Lignes, error in=R1.error out.
 7. R2.error out → registre droit. Les deux sorties Rectangle sont volontairement non câblées.
 8. Après la boucle, sortie finale du registre droit → sélecteur S4 et fil d'erreur S4. n et Lignes contournent la boucle par des fils pour entrer dans S4.
 9. S4.No Error : n entrant → sortie n ; Lignes entrant → sortie lignes ; erreur finale → sortie erreur. S4.Error est le cas zéro/vide/erreur défini en G5.1.
 10. Relier successivement les trois sorties S4 vers S3, S3 vers S2, S2 vers S1, S1 vers S0. Enregistrer.
- R1 vérifie toutes les largeurs de colonnes à la ligne0. R2 vérifie toutes les hauteurs de lignes à la colonne0. Toutes les combinaisons de cellules sont alors couvertes pour ces dimensions indépendantes. Après une erreur, les itérations restantes transmettent l'erreur sans refaire de travail ; les sous-VIs sont eux-mêmes protégés par leur cas Error.

G5.6 Tests des images fournies

Brancher LireImage.Bitmap sur ComputeRowsCols.Bitmap et chaîner les erreurs. Faire chaque essai séparément et noter les valeurs réellement observées.

Image fournie	Résultat géométrique attendu
Binairo_6x6.png	image454×431 ; n=6 ; sept traits X, sept traits Y ; aucune erreur
Binairo_8x8.png	image610×577 ; n=8 ; neuf traits X, neuf traits Y ; aucune erreur
Binairo_4x4_Bad.png	n=4 ; cinq traits par axe ; aucune erreur géométrique : sa contradiction relève ensuite de MATLAB
Binaro_5x6_Bad.png	six lignes et cinq colonnes : erreur6103, n=0, Lignes vide

Sur le6×6, ComputeCellRect(ligne0,colonne0) doit fournir gauche6,haut6,droite78,bas74, donc largeur72 et hauteur68. Ce crop intérieur est différent du fichier d'exemple Cell0.bin74×74, qui conserve une bordure. Ne pas les comparer comme des fichiers identiques.

G6. Essais géométriques sans image et extraction effective

G6.1 Construire un petit bitmap de test sans fichier externe

1. Dans un VI temporaire de test, construire un tableau Boolean2D de **hauteur38, largeur42** avec les deux boucles décrites ci-dessous.
2. Placer une For Loop extérieure N=38 et intérieure N=42. Indices : i extérieur=ligne ; i intérieur=colonne.
3. À l'intérieur, comparer colonne aux six limites pour produire (colonne>=0 ET colonne<2) OU (colonne>=20 ET colonne<22) OU (colonne>=40 ET colonne<42). Chaque intervalle utilise Greater Or Equal?, Less?, AND ; réunir les trois résultats avec Compound Arithmetic en mode OR, agrandi à trois entrées, ou deux OR successifs.
4. De même pour les lignes : [0,2), [18,20), [36,38).
5. OR des résultats vertical/horizontal → sortie auto-indexée de la boucle intérieure ; tableau1D obtenu → sortie auto-indexée de la boucle extérieure. Le bitmap testé est la **sortie des deux boucles**.
6. Bitmap construit → ComputeRowsCols. Attendu n=2, debutX0,20,40 ; finX2,22,42 ; debutY0,18,36 ; finY2,20,38. Chaque case intérieure18×16 respecte10..100. Les quatre côtés touchent les bords de l'image : aucun trait ne doit disparaître.
7. Pour tester une grille de **2 lignes et4 colonnes**, garder les intervalles Y, mettre largeur82 et N intérieur82, et utiliser cinq intervalles X [0,2), [20,22), [40,42), [60,62), [80,82). Attendu6103, n=0, Lignes vide. L'image rectangulaire précédente était valide ; ici c'est le **nombre de cases différent** qui est invalide.
8. Pour une grille3×3 impaire, utiliser dimension62×62 et quatre intervalles par axe [0,2), [20,22), [40,42), [60,62). Attendu6103.
9. Image entièrement FALSE38×42 →6102. Bitmap vide →6101. Une erreur entrante123 ressort123 avec n0/Lignes vide.
10. Tester séparément les dimensions de case9/10/100/101 dans ComputeCellRect comme G4.4. Ne pas garder ces VIs de test dans la livraison finale si elle doit contenir uniquement les fichiers nécessaires au programme.

G6.2 Extraire une case dans le futur VI de reconnaissance

1. ComputeCellRect reçoit ligne,colonne,Lignes et le cluster d'erreur courant.
2. Placer une Case sur son error out. Dans Error : pas d'extraction, pas d'écriture, pas d'appel OCR ; propager l'erreur.
3. Dans No Error : Unbundle By Name(Rectangle) donne gauche,haut,droite,bas.
4. Deux Subtract : droite–gauche → largeur ; bas–haut → hauteur.
5. Array Subset reçoit Bitmap original2D ; régler pour deux dimensions : index0=haut, length0=hauteur, index1=gauche, length1=largeur.
6. Vérifier Array Size du résultat : index0 doit donner hauteur, index1 largeur. Ce tableau reste Boolean2D avec TRUE noir.
7. Le sous-VI d'écriture du chapitre suivant le convertira en U8 et écrira largeur puis hauteur. **Ne pas transposer pour corriger l'ordre de l'en-tête.**

Le calcul suppose une grille alignée sur les axes et des lignes d'analyse à bord+10 qui ne traversent pas les chiffres, comme dans l'énoncé. Les données fournies et les cas limites ci-dessus doivent tous être essayés sur la VM. Une flèche exécutable et des résultats attendus dans un document ne remplacent pas ces essais.